

PSE-Descriptive Analysis

2026-09-12

Loading Necessary Libraries

```
library(readr)
library(dplyr)
library(ggplot2)
library(kableExtra)
library(ggtext)
```

I. 1st Stage of Cleaning: Removal of Pilot 2 Data

I removed Pilot 2 records from the main PSE dataset. I first read the Pilot 2 worker IDs. Then, for each of the seven main-data files, I removed every row with a matching `workerid` and saved the cleaned file using the same file name.

```
# Read the Pilot 2 worker IDs
pilot_ids <- read_csv(
  "/Users/taravat/Desktop/QP/QP/Data/Pipeline-Pilot2 Data/PSE-workerids.csv"
) %>%
  distinct(workerid)

# Folder containing the main-study files
main_folder <- "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Uncleaned data"

# Names of all seven files
file_names <- c(
  "PSE-condition.csv",
  "PSE-merged.csv",
  "PSE-subject_information.csv",
  "PSE-system.csv",
  "PSE-time_in_minutes.csv",
  "PSE-trials.csv",
  "PSE-workerids.csv"
)

# Remove Pilot 2 worker IDs from every file and replace the original files
for (file_name in file_names) {

  data_all <- read_csv(
    file.path(main_folder, file_name),
    show_col_types = FALSE
  )
```

```

data_clean <- data_all %>%
  anti_join(pilot_ids, by = "workerid")

write_csv(
  data_clean,
  file.path(main_folder, file_name)
)
}

```

II. 2nd Stage of Data Cleaning: Removal of Unapproved Participant Records

I compared the 360 approved participant IDs in the Prolific demographic file with the participant IDs in the Proliferate worker-ID records. This comparison identified six worker IDs that were not linked to an approved Prolific participant: 419, 400, 397, 220, 364, and 261. I removed all rows corresponding to these six worker IDs from PSE main dataset files to ensure that only records linked to approved Prolific participants remained. This left 357 worker-ID records in my PSE main dataset.

```

# Read the Prolific download and keep approved participants only
prolific_data <- read_csv(
  "/Users/taravat/Desktop/QP/PSE.csv",
  show_col_types = FALSE
) %>%
  filter(Status == "APPROVED") %>%
  select(
    prolific_participant_id = `Participant id`
  ) %>%
  distinct()

# Read the worker-ID file
workerids <- read_csv(
  "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Uncleaned data/PSE-workerids.csv",
  show_col_types = FALSE
) %>%
  distinct(workerid, prolific_participant_id)

# Show only the number of participants in each file
comparison_summary <- tibble(
  file = c(
    "Approved participants in Prolific file",
    "Participants in worker-ID file"
  ),
  number_of_participants = c(
    nrow(prolific_data),
    nrow(workerids)
  )
)

knitr::kable(
  comparison_summary,
  col.names = c("File", "Number of participants"),

```

```
caption = "Number of participants in each file"
)
```

Table 1: Number of participants in each file

File	Number of participants
Approved participants in Prolific file	360
Participants in worker-ID file	357

```
# Find worker-ID records that are not in the approved Prolific file
extra_workerids <- workerids %>%
  anti_join(
    prolific_data,
    by = "prolific_participant_id"
  )

# Find approved Prolific participants missing from the worker-ID file
missing_workerids <- prolific_data %>%
  anti_join(
    workerids,
    by = "prolific_participant_id"
  )

# Display only the number of missing records, not their Prolific IDs
missing_summary <- tibble(
  issue = "Approved participants missing from the worker-ID file",
  number_of_participants = nrow(missing_workerids)
)

## Removing Worker IDs Not Found in the Approved Prolific File

# Keep only the worker IDs that are not in the approved Prolific file
extra_ids <- extra_workerids %>%
  select(workerid)

# Remove these six records from the worker-ID file
workerids_clean <- workerids %>%
  anti_join(
    extra_ids,
    by = "workerid"
  )

write_csv(
  workerids_clean,
  "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Uncleaned data/PSE-workerids.csv"
)

# Folder containing the other main-data files
main_folder <- "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Uncleaned data"

# Files from which the six worker IDs will be removed
file_names <- c(
```

```

"PSE-merged.csv",
"PSE-subject_information.csv",
"PSE-system.csv",
"PSE-time_in_minutes.csv",
"PSE-trials.csv"
)

# Remove the six worker IDs and replace each file
removal_summary <- tibble()

for (file_name in file_names) {

  data_all <- read_csv(
    file.path(main_folder, file_name),
    show_col_types = FALSE
  )

  data_clean <- data_all %>%
    anti_join(
      extra_ids,
      by = "workerid"
    )

  removal_summary <- bind_rows(
    removal_summary,
    tibble(
      file = file_name,
      rows_removed = nrow(data_all) - nrow(data_clean)
    )
  )

  write_csv(
    data_clean,
    file.path(main_folder, file_name)
  )
}

```

III. 3rd Stage of Data Exclusion: Control-Item Performance and Response Variability

Following **Degen and Tonhauser (2022)** and consistent with the exclusion criteria reported in my **IRB protocol**, participants will be excluded if their **mean response across the six control items is more than two standard deviations above the group mean** or if the **variance of their responses across all trials is more than two standard deviations below the group mean variance**.

For each of the **357 participants** remaining after the first two cleaning stages, I calculated two values: a control mean and a response variance. The control mean is the participant's average response across the six control items, for which the expected answer is close to No (0); for example, a control mean of 0.99 means that the participant responded close to Yes on nearly all six controls. Response variance measures how much that participant's ratings changed across all 18 trials: a very low value, such as 0.01, means they gave almost the same slider response repeatedly, whereas a larger value, such as 0.20, indicates that their responses varied across items. I then compared each participant's control mean and response variance with the corresponding

group mean and group standard deviation across all participants. Participants were excluded if their control mean was more than 2 SD above the group mean or if their response variance was more than 2 SD below the group mean variance. In the current data, the excluded participants met the control-response criterion; no participant met the low-variance criterion.

```
# Folders for input and cleaned output
uncleaned_folder <- "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Uncleaned data"

cleaned_folder <- "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Cleaned data"

# Read the dataset after the first two stages of cleaning
data_raw <- read_csv(
  file.path(uncleaned_folder, "PSE-merged.csv"),
  show_col_types = FALSE
)

# Create the unique participant identifier
data_analysis <- data_raw %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_"),
    response = as.numeric(response)
  )

# Calculate each participant's control mean and overall response variance
participant_quality <- data_analysis %>%
  mutate(
    is_control = grepl("^control", item_id)
  ) %>%
  group_by(participant_id, workerid, time_in_minutes) %>%
  summarise(
    control_mean = mean(response[is_control]),
    response_variance = var(response),
    .groups = "drop"
  )

# Calculate the exclusion cutoffs
control_cutoff <- mean(participant_quality$control_mean) +
  2 * sd(participant_quality$control_mean)

variance_cutoff <- mean(participant_quality$response_variance) -
  2 * sd(participant_quality$response_variance)

# Show the group values and exclusion cutoffs
cutoff_summary <- tibble(
  criterion = c(
    "Control-item responses",
    "Overall response variance"
  ),
  group_mean = c(
    mean(participant_quality$control_mean),
    mean(participant_quality$response_variance)
  ),
  group_sd = c(
```

```

    sd(participant_quality$control_mean),
    sd(participant_quality$response_variance)
  ),
  exclusion_cutoff = c(
    control_cutoff,
    variance_cutoff
  ),
  exclude_if = c(
    "Above the cutoff",
    "Below the cutoff"
  )
)

knitr::kable(
  cutoff_summary,
  digits = 3,
  col.names = c(
    "Criterion",
    "Group mean",
    "Group SD",
    "Exclusion cutoff",
    "Participant is excluded if"
  ),
  caption = "Group values and exclusion cutoffs"
)

```

Table 2: Group values and exclusion cutoffs

Criterion	Group mean	Group SD	Exclusion cutoff	Participant is excluded if
Control-item responses	0.180	0.244	0.667	Above the cutoff
Overall response variance	0.109	0.060	-0.011	Below the cutoff

```

# Identify participants who meet either exclusion criterion
quality_exclusions <- participant_quality %>%
  mutate(
    exclusion_reason = case_when(
      control_mean > control_cutoff &
        response_variance < variance_cutoff ~
          "Control responses and low response variance",

      control_mean > control_cutoff ~
          "Control responses",

      response_variance < variance_cutoff ~
          "Low response variance",

      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(exclusion_reason))

```

```

# Show the participant IDs to be excluded
knitr::kable(
  quality_exclusions %>%
    select(
      participant_id,
      workerid,
      control_mean,
      response_variance,
      exclusion_reason
    ),
  digits = 3,
  col.names = c(
    "Participant ID",
    "Worker ID",
    "Control mean",
    "Response variance",
    "Exclusion reason"
  ),
  caption = "Participants excluded based on control responses and response variability"
)

```

Table 3: Participants excluded based on control responses and response variability

Participant ID	Worker ID	Control mean	Response variance	Exclusion reason
104_4.07925	104	0.988	0.084	Control responses
123_8.798616666666667	123	0.977	0.088	Control responses
131_5.30125	131	0.970	0.199	Control responses
140_4.838166666666667	140	0.980	0.190	Control responses
189_4.877066666666667	189	0.997	0.014	Control responses
207_3.473116666666667	207	0.752	0.036	Control responses
228_4.963933333333333	228	0.747	0.020	Control responses
231_2.897783333333333	231	0.862	0.093	Control responses
232_5.7731	232	0.848	0.068	Control responses
272_2.70885	272	0.982	0.035	Control responses
304_7.389083333333333	304	0.973	0.169	Control responses
307_5.637683333333333	307	0.982	0.171	Control responses
354_7.1107	354	0.743	0.050	Control responses
403_12.621116666666667	403	0.863	0.122	Control responses
417_5.2179	417	0.905	0.156	Control responses
420_11.386683333333333	420	0.695	0.056	Control responses
425_9.447766666666667	425	1.000	0.182	Control responses
430_3.427166666666667	430	0.908	0.007	Control responses
47_6.43135	47	0.995	0.073	Control responses
65_5.35195	65	0.977	0.129	Control responses

```

# Worker IDs to exclude
excluded_workerids <- quality_exclusions %>%
  distinct(workerid)

# Files to clean
file_names <- c(

```

```

"PSE-condition.csv",
"PSE-merged.csv",
"PSE-subject_information.csv",
"PSE-system.csv",
"PSE-time_in_minutes.csv",
"PSE-trials.csv",
"PSE-workerids.csv"
)

# Read each uncleaned file, remove excluded worker IDs,
# and save the cleaned version in the Cleaned data folder
for (file_name in file_names) {

  data_all <- read_csv(
    file.path(uncleaned_folder, file_name),
    show_col_types = FALSE
  )

  data_clean <- data_all %>%
    anti_join(excluded_workerids, by = "workerid")

  write_csv(
    data_clean,
    file.path(cleaned_folder, file_name)
  )
}

```

Note on participant identifiers: For participant-level quality checks and summaries, I created a participant identifier by *combining the worker ID with the recorded completion time (time_in_minutes)*. This was necessary because the dataset contained repeated worker IDs #344; using the worker ID alone would not distinguish these two observations. For saving the cleaned data files, exclusions were applied using the worker ID because worker ID #344 did not appear among the excluded participants in Table 3, I used the worker ID to remove excluded records from the cleaned data files.

IV. 4th Stage of Cleaning: Exclusion of Non-Native English Speakers

Native English speaker status was an eligibility criterion, and the study was restricted to Prolific participants who met this prescreening requirement. However, three participants self-identified a language other than English as their native language in response to the question, “Native language(s) (spoken at home when you were a child).” In this stage, I excluded the data associated with these three participants from the cleaned dataset to ensure that the final analytic sample met the study’s native-English eligibility criterion.

```

# Read the cleaned merged file in Cleaned data folder
data_raw <- read_csv(
  file.path(cleaned_folder, "PSE-merged.csv"),
  show_col_types = FALSE
)

# Create the participant identifier
data_analysis <- data_raw %>%

```

```

mutate(
  participant_id = paste(workerid, time_in_minutes, sep = "_")
)

# Keep one row for each participant
demographics <- data_analysis %>%
  distinct(participant_id, .keep_all = TRUE)

# Identify participants who did not list English as a native language
non_english_ids <- demographics %>%
  mutate(
    language = tolower(subject_information.language),
    language_group = case_when(
      is.na(language) ~ "Not provided",
      language %in% c(
        "english",
        "enlish",
        "englih",
        "american english"
      ) ~ "English only",
      grepl("english", language) ~ "English and another language",
      TRUE ~ "No English listed"
    )
  ) %>%
  filter(language_group == "No English listed") %>%
  distinct(workerid)

# Show the worker IDs that will be removed
knitr::kable(
  non_english_ids,
  col.names = "Worker ID",
  caption = "Worker IDs excluded because English was not listed as a native language"
)

```

Table 4: Worker IDs excluded because English was not listed as a native language

Worker ID
291
168
54

```

# Files to update
file_names <- c(
  "PSE-condition.csv",
  "PSE-merged.csv",
  "PSE-subject_information.csv",
  "PSE-system.csv",
  "PSE-time_in_minutes.csv",
  "PSE-trials.csv"
)

```

```

# Remove these worker IDs and replace the Stage 3 cleaned files
for (file_name in file_names) {

  data_all <- read_csv(
    file.path(cleaned_folder, file_name),
    show_col_types = FALSE
  )

  data_clean <- data_all %>%
    anti_join(
      non_english_ids,
      by = "workerid"
    )

  write_csv(
    data_clean,
    file.path(cleaned_folder, file_name)
  )
}

```

V. Mean and Median Completion Time

I calculated the median as well as the mean because the median is less affected by extremely fast or slow completion times than the mean.

```

# Read the dataset after all cleaning stages
data_raw <- read_csv(
  file.path(cleaned_folder, "PSE-merged.csv"),
  show_col_types = FALSE
)

# Create the participant identifier again
data_analysis <- data_raw %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_")
  )

completion_time_summary <- data_analysis %>%
  distinct(participant_id, .keep_all = TRUE) %>%
  summarise(
    mean_completion_time = round(
      mean(time_in_minutes, na.rm = TRUE),
      2
    ),
    median_completion_time = round(
      median(time_in_minutes, na.rm = TRUE),
      2
    )
  )

knitr::kable(
  completion_time_summary,

```

```
col.names = c(
  "Mean completion time (minutes)",
  "Median completion time (minutes)"
),
caption = "Completion time summary"
)
```

Table 5: Completion time summary

Mean completion time (minutes)	Median completion time (minutes)
5.92	5.33

VI. Demographic Data

All analyses reported from this point onward are based on the final cleaned dataset after applying all exclusion criteria.

A. Number of Participants

Table 6 below reports the number of unique worker IDs and unique participant identifiers in the final cleaned dataset. As noted in Section III, worker ID #344 was assigned to two different participants and therefore had 36 trial rows rather than the expected 18 rows (see Table 7). To distinguish these two participants, I created a participant identifier by combining **workerid** and **time_in_minutes** (see Table 8). This identifier was used for participant-level summaries and analyses so that the two sessions associated with worker ID #344 were treated as separate participants. The final cleaned dataset includes **334 participants**.

```
# Read the final cleaned merged file
data_raw <- read_csv(
  file.path(cleaned_folder, "PSE-merged.csv"),
  show_col_types = FALSE
)

# Create the participant identifier
data_analysis <- data_raw %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_")
  )

# Number of unique worker IDs and unique participants
participant_summary <- data_analysis %>%
  summarise(
    unique_worker_ids = n_distinct(workerid),
    unique_participants = n_distinct(participant_id)
  )

knitr::kable(
  participant_summary,
  col.names = c(
    "Unique worker IDs",
    "Unique participants"
  )
)
```

```

),
caption = "Number of participants in the final cleaned dataset"
)

```

Table 6: Number of participants in the final cleaned dataset

Unique worker IDs	Unique participants
333	334

```

# Show any worker ID with more than 18 trial rows
repeated_workerids <- data_analysis %>%
  count(
    workerid,
    name = "Number of trial rows"
  ) %>%
  filter(`Number of trial rows` > 18)

knitr::kable(
  repeated_workerids,
  col.names = c(
    "Worker ID",
    "Number of trial rows"
  ),
  caption = "Worker IDs with more than 18 trial rows"
)

```

Table 7: Worker IDs with more than 18 trial rows

Worker ID	Number of trial rows
344	36

```

# Show the two participant identifiers for worker ID 344
data_analysis %>%
  filter(workerid == 344) %>%
  distinct(participant_id) %>%
  knitr::kable(
    col.names = "Participant ID",
    caption = "Participant identifiers created for worker ID 344"
  )

```

Table 8: Participant identifiers created for worker ID 344

Participant ID
344_7.717533333333333
344_4.479683333333333

B.Age Distribution

```
# Keep one row for each participant
demographics <- data_analysis %>%
  distinct(participant_id, .keep_all = TRUE) %>%
  mutate(
    age = as.numeric(subject_information.age)
  )

# Age summary, including nonresponses
age_summary<-demographics %>%
  summarise(
    n_age = sum(!is.na(age)),
    no_age_response = sum(is.na(age)),
    mean_age = mean(age, na.rm = TRUE),
    sd_age = sd(age, na.rm = TRUE),
    median_age = median(age, na.rm = TRUE),
    minimum_age = min(age, na.rm = TRUE),
    maximum_age = max(age, na.rm = TRUE)
  )
knitr::kable(
  age_summary,
  col.names = c(
    "Age responses",
    "No response",
    "Mean age",
    "SD",
    "Median age",
    "Minimum age",
    "Maximum age"
  ),
  caption = "Age summary of participants"
)
```

Table 9: Age summary of participants

Age responses	No response	Mean age	SD	Median age	Minimum age	Maximum age
334	0	42.71557	13.34594	40	18	80

```
# Age Plot
age_ranges <- demographics %>%
  filter(!is.na(age)) %>%
  mutate(
    age_range = cut(
      age,
      breaks = seq(20, 85, by = 5),
      right = FALSE,
      labels = c(
        "20-24", "25-29", "30-34",
        "35-39", "40-44", "45-49",

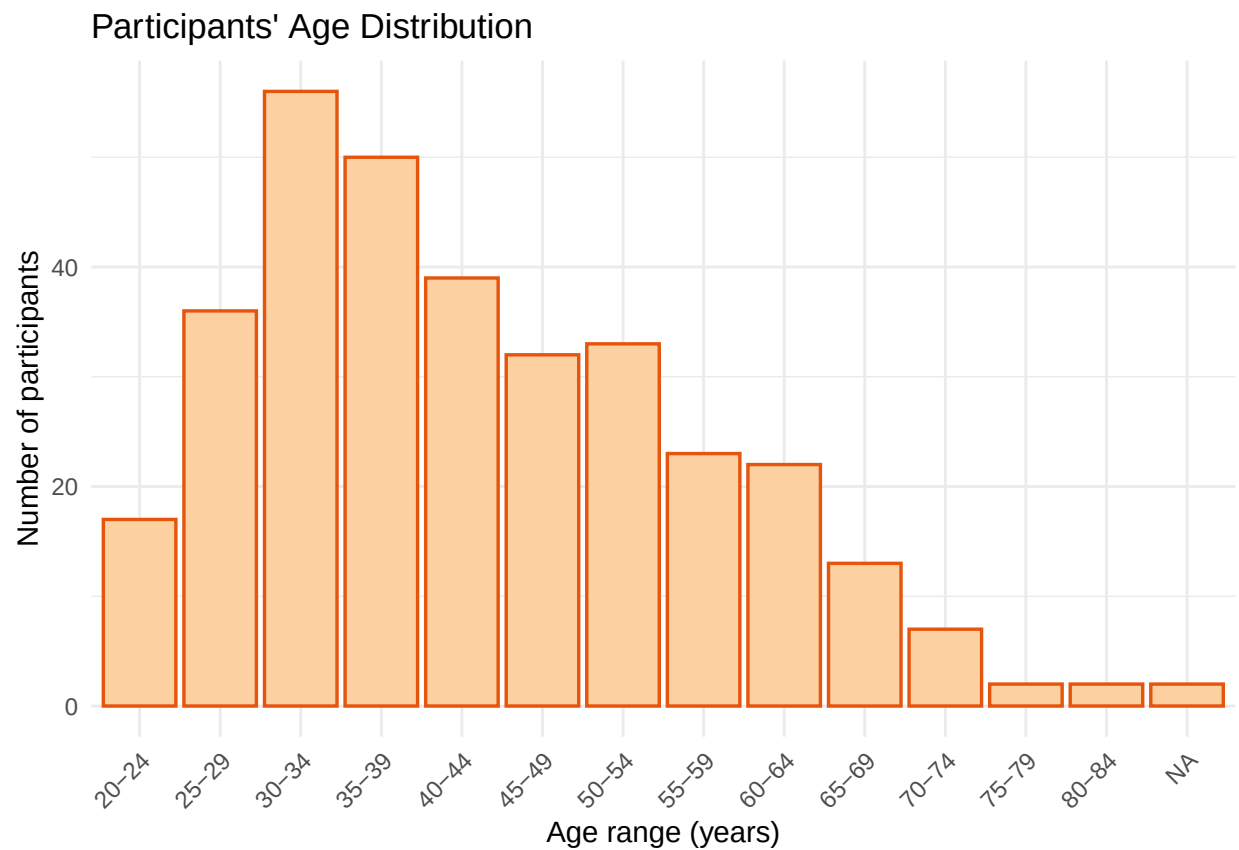
```

```

    "50-54", "55-59", "60-64",
    "65-69", "70-74", "75-79",
    "80-84"
  )
)
) %>%
count(age_range)

ggplot(age_ranges, aes(x = age_range, y = n)) +
  geom_col(
    fill = "#FDD0A2",
    color = "#E6550D",
    linewidth = 0.6
  ) +
  labs(
    title = "Participants' Age Distribution",
    x = "Age range (years)",
    y = "Number of participants"
  ) +
  theme_minimal() +
  theme(
    axis.text.x = element_text(angle = 45, hjust = 1)
  )
)

```



C. Gender Distribution

```
gender_summary <- demographics %>%
  mutate(
    gender = case_when(
      subject_information.gender == "f" ~ "Female",
      subject_information.gender == "m" ~ "Male",
      subject_information.gender == "o" ~ "Other",
      TRUE ~ "No response"
    )
  ) %>%
  count(gender) %>%
  mutate(
    percent = round(100 * n / sum(n), 1)
  )

knitr::kable(
  gender_summary,
  col.names = c("Gender", "Number of participants", "Percentage"),
  caption = "Gender distribution of participants"
)
```

Table 10: Gender distribution of participants

Gender	Number of participants	Percentage
Female	187	56.0
Male	142	42.5
Other	5	1.5

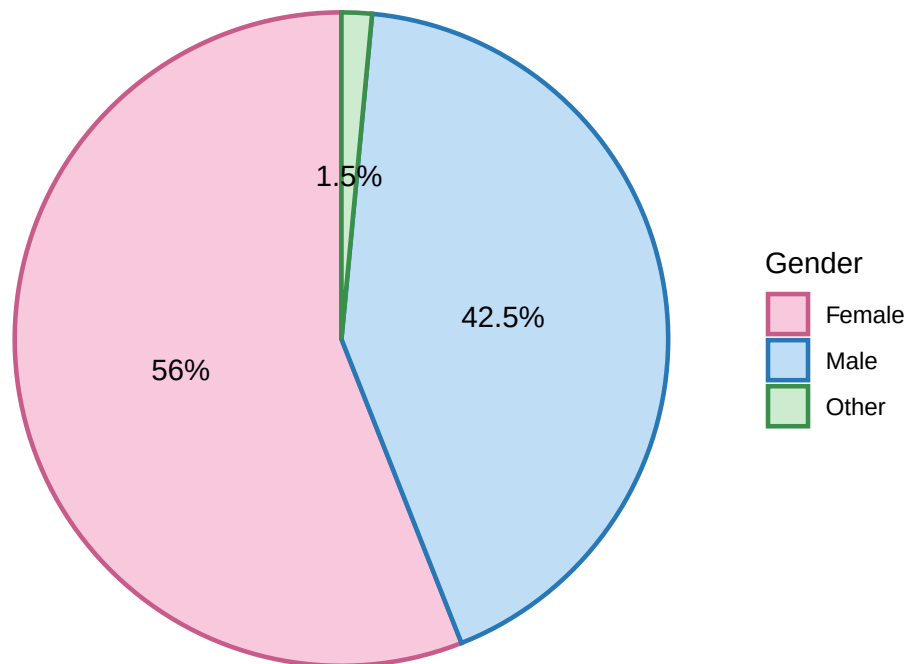
```
ggplot(
  gender_summary,
  aes(x = "", y = n, fill = gender, color = gender)
) +
  geom_col(
    width = 1,
    linewidth = 0.8
  ) +
  coord_polar(theta = "y") +
  geom_text(
    aes(label = paste0(percent, "%")),
    position = position_stack(vjust = 0.5),
    color = "black"
  ) +
  scale_fill_manual(
    values = c(
      "Female" = "#F8C8DC",
      "Male" = "#BFDDE5",
      "Other" = "#CDECCF",
      "No response" = "#D9D9D9"
    )
  ) +
```

```

scale_color_manual(
  values = c(
    "Female" = "#C75C8A",
    "Male" = "#2878B5",
    "Other" = "#3A8F4B",
    "No response" = "#777777"
  )
) +
labs(
  title = "Gender Distribution",
  fill = "Gender",
  color = "Gender"
) +
theme_void()

```

Gender Distribution



D. Education Distribution

```

education_summary <- demographics %>%
  mutate(
    education = case_when(
      is.na(subject_information.education) ~ "No response",
      TRUE ~ subject_information.education
    ),

```

```

education = factor(
  education,
  levels = c(
    "Some High School",
    "Graduated High School",
    "Some College",
    "Graduated College",
    "Hold a higher degree",
    "No response"
  )
)
) %>%
count(education) %>%
mutate(
  percent = round(100 * n / sum(n), 1)
)

knitr::kable(
  education_summary,
  col.names = c(
    "Education level",
    "Number of participants",
    "Percentage"
  ),
  caption = "Education level of participants"
)

```

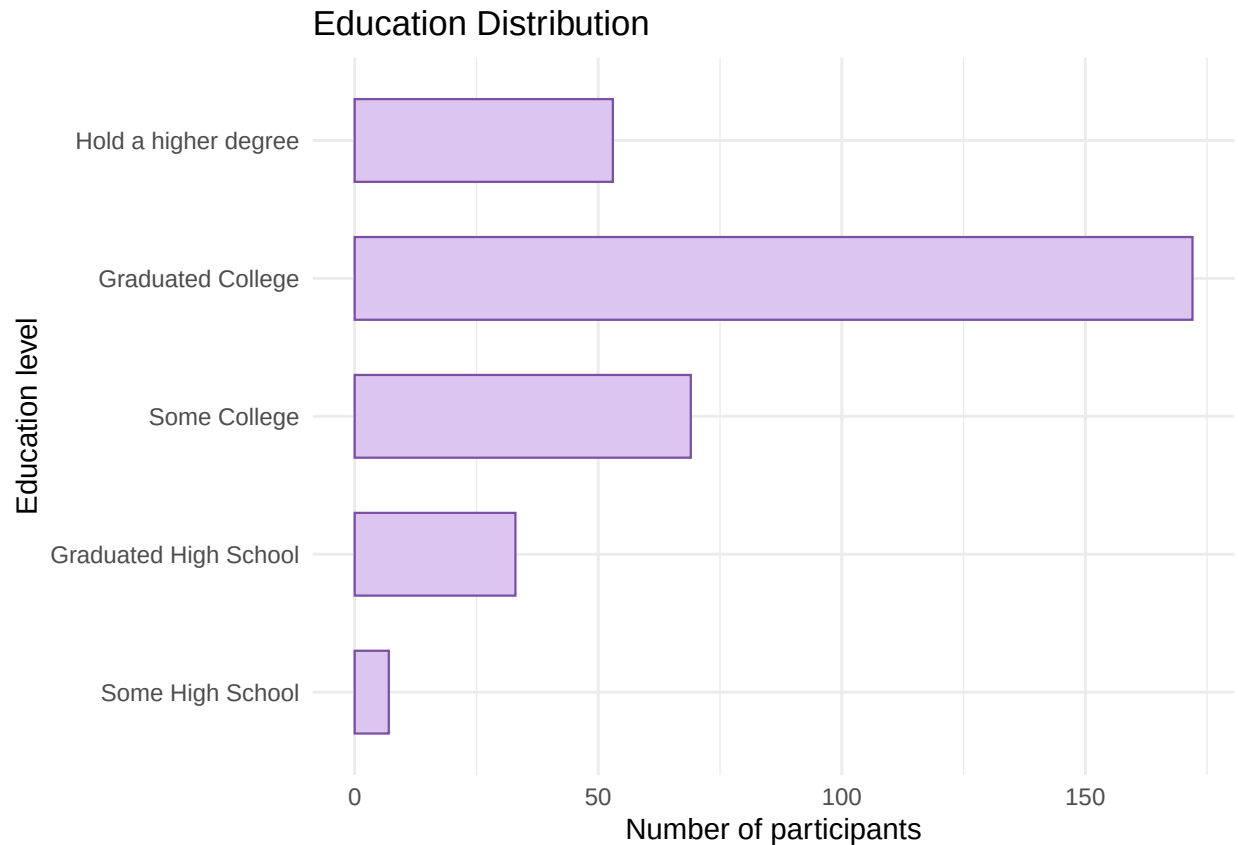
Table 11: Education level of participants

Education level	Number of participants	Percentage
Some High School	7	2.1
Graduated High School	33	9.9
Some College	69	20.7
Graduated College	172	51.5
Hold a higher degree	53	15.9

```

ggplot(education_summary, aes(x = n, y = education)) +
  geom_col(
    fill = "#DCC6F0",
    color = "#7B4FA3",
    linewidth = 0.4,
    width = 0.6
  ) +
  labs(
    title = "Education Distribution",
    x = "Number of participants",
    y = "Education level"
  ) +
  theme_minimal()

```



E. American English Status

```
american_summary <- demographics %>%
  mutate(
    american_english = case_when(
      subject_information.american == "y" ~ "Yes",
      subject_information.american == "n" ~ "No",
      TRUE ~ "Not provided"
    )
  ) %>%
  count(american_english) %>%
  mutate(
    percent = round(100 * n / sum(n), 1)
  )

knitr::kable(
  american_summary,
  col.names = c(
    "American English",
    "Number of participants",
    "Percentage"
  ),
  caption = "American English status of participants"
)
```

Table 12: American English status of participants

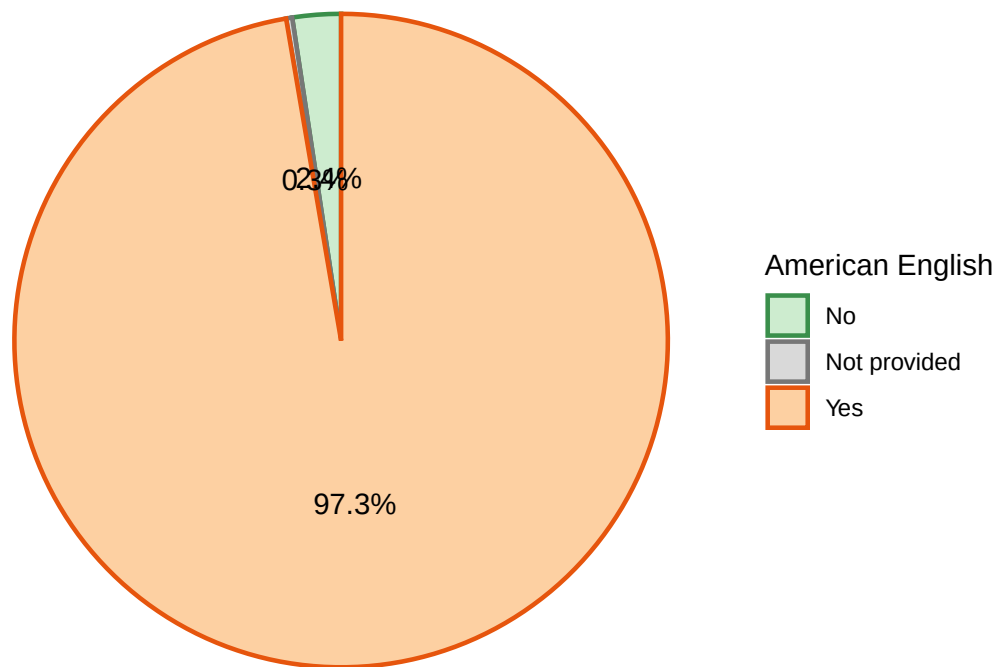
American English	Number of participants	Percentage
No	8	2.4
Not provided	1	0.3
Yes	325	97.3

```

ggplot(
  american_summary,
  aes(x = "", y = n, fill = american_english, color = american_english)
) +
  geom_col(
    width = 1,
    linewidth = 0.8
  ) +
  coord_polar(theta = "y") +
  geom_text(
    aes(label = paste0(percent, "%")),
    position = position_stack(vjust = 0.5),
    color = "black"
  ) +
  scale_fill_manual(
    values = c(
      "Yes" = "#FDD0A2",
      "No" = "#CDECCF",
      "Not provided" = "#D9D9D9"
    )
  ) +
  scale_color_manual(
    values = c(
      "Yes" = "#E6550D",
      "No" = "#3A8F4B",
      "Not provided" = "#777777"
    )
  ) +
  labs(
    title = "American English Status",
    fill = "American English",
    color = "American English"
  ) +
  theme_void()

```

American English Status



F. Native Language(s)

```
language_summary <- demographics %>%
  mutate(
    language_response = subject_information.language,
    language = tolower(trimws(language_response)),

    language_group = case_when(
      is.na(language) | language == "" ~ "Not provided",

      language %in% c(
        "english",
        "enlish",
        "engliah",
        "american english"
      ) ~ "English / American English",

      grepl("english", language) ~
        "Bilingual: English and another language"
    )
  ) %>%
  count(language_group) %>%
  mutate(
    percent = round(100 * n / sum(n), 1)
```

```

)

knitr::kable(
  language_summary,
  col.names = c(
    "Native language group",
    "Number of participants",
    "Percentage"
  ),
  caption = "Native language background of participants"
)

```

Table 13: Native language background of participants

Native language group	Number of participants	Percentage
Bilingual: English and another language	15	4.5
English / American English	312	93.4
Not provided	7	2.1

```

# Separate table listing the bilingual language responses
bilingual_details <- demographics %>%
  mutate(
    language_response = subject_information.language,
    language = tolower(trimws(language_response))
  ) %>%
  filter(
    !is.na(language),
    grepl("english", language),
    !language %in% c(
      "english",
      "enlish",
      "engliah",
      "american english"
    )
  ) %>%
  count(language_response, name = "Number of participants")

knitr::kable(
  bilingual_details,
  col.names = c(
    "Reported native-language response",
    "Number of participants"
  ),
  caption = "Reported native languages of bilingual participants"
)

```

Table 14: Reported native languages of bilingual participants

Reported native-language response	Number of participants
Albanian, English	1

Reported native-language response	Number of participants
Dutch, English (before5)	1
English and Chinese	1
English and Spanish	1
English, Chinese	1
English, Japanese	1
English, Korean	1
English, Spanish	2
English, Vietnamese	1
English, cantonese	1
English, some Russian, some Hebrew, some French	1
Serbian, English	1
Vietnamese, English	1
english and spanish	1

VII.Trial Characteristics and Distribution

A. Overall Distribution of Trial Characteristics

```

# Read the final cleaned merged file
data_final <- read_csv(
  "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Cleaned data/PSE-merged.csv",
  show_col_types = FALSE
)

cumulative_summary <- bind_rows(

  # Control first, then C triggers, then NonProj triggers
  data_final %>%
    mutate(
      trigger_group = case_when(
        trigger == "control" | is.na(trigger) ~ "Control",
        TRUE ~ trigger_class
      ),
      category = case_when(
        trigger == "control" | is.na(trigger) ~ "Control",
        TRUE ~ as.character(trigger)
      )
    ) %>%
    count(trigger_group, category, name = "n") %>%
    mutate(variable = "Trigger"),

  # Scale
  data_final %>%
    filter(!is.na(scale)) %>%
    count(category = as.character(scale), name = "n") %>%
    mutate(variable = "Scale"),

  # Condition
  data_final %>%

```

```

    filter(!is.na(condition)) %>%
    count(category = as.character(condition), name = "n") %>%
    mutate(variable = "Condition"),

# Slide number
data_final %>%
  count(
    category = as.character(slide_number_in_experiment),
    name = "n"
  ) %>%
  mutate(variable = "Slide number"),

# Matrix-subject names only
data_final %>%
  filter(!is.na(matrix_subject)) %>%
  count(category = matrix_subject, name = "n") %>%
  mutate(variable = "Matrix subject")

) %>%
mutate(
  variable = factor(
    variable,
    levels = c(
      "Trigger",
      "Scale",
      "Condition",
      "Slide number",
      "Matrix subject"
    )
  ),
  trigger_group = factor(
    trigger_group,
    levels = c("Control", "C", "NonProj")
  ),
  percent = round(100 * n / nrow(data_final), 1)
) %>%
arrange(variable, trigger_group, category) %>%
select(variable, category, n, percent)

knitr::kable(
  cumulative_summary,
  format = "latex",
  booktabs = TRUE,
  col.names = c(
    "Variable",
    "Category",
    "Number of trial rows",
    "Percentage of trial rows"
  ),
  caption = "Cumulative description of the final dataset"
) %>%
kableExtra::kable_styling(
  latex_options = "HOLD_position",

```

```
font_size = 7
```

```
)
```

Table 15: Cumulative description of the final dataset

Variable	Category	Number of trial rows	Percentage of trial rows
Trigger	Control	2004	33.3
Trigger	acknowledge	334	5.6
Trigger	confirm	334	5.6
Trigger	hear	334	5.6
Trigger	inform_Sam	334	5.6
Trigger	prove	334	5.6
Trigger	say	334	5.6
Trigger	suggest	334	5.6
Trigger	discover	334	5.6
Trigger	establish	334	5.6
Trigger	know	334	5.6
Trigger	reveal	334	5.6
Trigger	think	334	5.6
Scale	or	2004	33.3
Scale	some	2004	33.3
Condition	lb	2004	33.3
Condition	ub	2004	33.3
Slide number	10	334	5.6
Slide number	11	334	5.6
Slide number	12	334	5.6
Slide number	13	334	5.6
Slide number	14	334	5.6
Slide number	15	334	5.6
Slide number	16	334	5.6
Slide number	17	334	5.6
Slide number	18	334	5.6
Slide number	19	334	5.6
Slide number	20	334	5.6
Slide number	21	334	5.6
Slide number	4	334	5.6
Slide number	5	334	5.6
Slide number	6	334	5.6
Slide number	7	334	5.6
Slide number	8	334	5.6
Slide number	9	334	5.6
Matrix subject	Amanda	334	5.6
Matrix subject	Andrew	334	5.6
Matrix subject	Barbara	334	5.6
Matrix subject	Brian	334	5.6
Matrix subject	Janice	334	5.6
Matrix subject	Justin	334	5.6
Matrix subject	Kevin	334	5.6
Matrix subject	Melissa	334	5.6
Matrix subject	Patrick	334	5.6
Matrix subject	Rachel	334	5.6
Matrix subject	Scott	334	5.6
Matrix subject	Tina	334	5.6

B.Predicate-and-Scale Distribution by Factivity Group

```
# Number of predicate-and-scale combinations by factivity group
factivity_scale_summary <- data_final %>%
  filter(!is.na(scale)) %>%
  mutate(
    factivity = case_when(
      trigger %in% c("discover", "know", "reveal") ~
        "Canonically factive",

      trigger %in% c(
        "prove", "confirm", "establish",
        "acknowledge", "hear", "inform_Sam"
      ) ~
        "Optionally factive",

      trigger %in% c("think", "suggest", "say") ~
        "Non-factive"
    ),

    predicate_name = case_when(
      trigger == "inform_Sam" ~ "inform",
      TRUE ~ as.character(trigger)
    ),

    predicate_scale = paste(
      predicate_name,
      scale,
      sep = " + "
    )
  ) %>%
  count(
    factivity,
    predicate_scale,
    name = "n"
  ) %>%
  mutate(
    percent = round(100 * n / nrow(data_final), 1),

    factivity = factor(
      factivity,
      levels = c(
        "Canonically factive",
        "Optionally factive",
        "Non-factive"
      )
    ),

    predicate_scale = factor(
      predicate_scale,
      levels = c(
        "discover + some",
        "discover + or",

```

```

      "know + some",
      "know + or",
      "reveal + some",
      "reveal + or",

      "prove + some",
      "prove + or",
      "confirm + some",
      "confirm + or",
      "establish + some",
      "establish + or",
      "acknowledge + some",
      "acknowledge + or",
      "hear + some",
      "hear + or",
      "inform + some",
      "inform + or",

      "think + some",
      "think + or",
      "suggest + some",
      "suggest + or",
      "say + some",
      "say + or"
    )
  )
) %>%
arrange(
  factivity,
  predicate_scale
) %>%
select(
  factivity,
  predicate_scale,
  n,
  percent
)

knitr::kable(
  factivity_scale_summary,
  format = "latex",
  booktabs = TRUE,
  col.names = c(
    "Factivity",
    "Predicate + scale",
    "Number of trial rows",
    "Percentage of trial rows"
  ),
  caption = "Distribution of predicate-and-scale combinations by factivity group"
) %>%
kableExtra::collapse_rows(
  columns = 1,
  valign = "top"
)

```

```

) %>%
kableExtra::kable_styling(
  latex_options = "HOLD_position",
  font_size = 8
)

```

Table 16: Distribution of predicate-and-scale combinations by factivity group

Factivity	Predicate + scale	Number of trial rows	Percentage of trial rows
Canonically factive	discover + some	162	2.7
	discover + or	172	2.9
	know + some	169	2.8
	know + or	165	2.7
	reveal + some	170	2.8
	reveal + or	164	2.7
Optionally factive	prove + some	165	2.7
	prove + or	169	2.8
	confirm + some	151	2.5
	confirm + or	183	3.0
	establish + some	176	2.9
	establish + or	158	2.6
	acknowledge + some	158	2.6
	acknowledge + or	176	2.9
	hear + some	178	3.0
	hear + or	156	2.6
	inform + some	170	2.8
	inform + or	164	2.7
Non-factive	think + some	146	2.4
	think + or	188	3.1
	suggest + some	174	2.9
	suggest + or	160	2.7
	say + some	185	3.1
	say + or	149	2.5

C.Predicate, Scale, and Boundedness Distribution by Factivity Group

```
# Number of predicate, scale, and boundedness combinations by factivity group
factivity_scale_condition_summary <- data_final %>%
  filter(
    !is.na(scale),
    !is.na(condition)
  ) %>%
  mutate(
    factivity = case_when(
      trigger %in% c("discover", "know", "reveal") ~
        "Canonically factive",

      trigger %in% c(
        "prove", "confirm", "establish",
        "acknowledge", "hear", "inform_Sam"
      ) ~
        "Optionally factive",

      trigger %in% c("think", "suggest", "say") ~
        "Non-factive"
    ),

    predicate_name = case_when(
      trigger == "inform_Sam" ~ "inform",
      TRUE ~ as.character(trigger)
    ),

    predicate_scale_boundedness = paste(
      predicate_name,
      scale,
      condition,
      sep = " + "
    )
  ) %>%
  count(
    factivity,
    predicate_scale_boundedness,
    name = "n"
  ) %>%
  mutate(
    percent = round(100 * n / nrow(data_final), 1),

    factivity = factor(
      factivity,
      levels = c(
        "Canonically factive",
        "Optionally factive",
        "Non-factive"
      )
    ),

    predicate_scale_boundedness = factor(
```

```

predicate_scale_boundedness,
levels = c(
  "discover + some + ub",
  "discover + some + lb",
  "discover + or + ub",
  "discover + or + lb",
  "know + some + ub",
  "know + some + lb",
  "know + or + ub",
  "know + or + lb",
  "reveal + some + ub",
  "reveal + some + lb",
  "reveal + or + ub",
  "reveal + or + lb",

  "prove + some + ub",
  "prove + some + lb",
  "prove + or + ub",
  "prove + or + lb",
  "confirm + some + ub",
  "confirm + some + lb",
  "confirm + or + ub",
  "confirm + or + lb",
  "establish + some + ub",
  "establish + some + lb",
  "establish + or + ub",
  "establish + or + lb",
  "acknowledge + some + ub",
  "acknowledge + some + lb",
  "acknowledge + or + ub",
  "acknowledge + or + lb",
  "hear + some + ub",
  "hear + some + lb",
  "hear + or + ub",
  "hear + or + lb",
  "inform + some + ub",
  "inform + some + lb",
  "inform + or + ub",
  "inform + or + lb",

  "think + some + ub",
  "think + some + lb",
  "think + or + ub",
  "think + or + lb",
  "suggest + some + ub",
  "suggest + some + lb",
  "suggest + or + ub",
  "suggest + or + lb",
  "say + some + ub",
  "say + some + lb",
  "say + or + ub",
  "say + or + lb"
)

```

```

    )
  ) %>%
  arrange(
    factivity,
    predicate_scale_boundedness
  ) %>%
  select(
    factivity,
    predicate_scale_boundedness,
    n,
    percent
  )

knitr::kable(
  factivity_scale_condition_summary,
  format = "latex",
  booktabs = TRUE,
  col.names = c(
    "Factivity",
    "Predicate + scale + boundedness",
    "Number of trial rows",
    "Percentage of trial rows"
  ),
  caption = "Distribution of predicate, scale, and boundedness combinations by factivity group"
) %>%
kableExtra::collapse_rows(
  columns = 1,
  valign = "top"
) %>%
kableExtra::kable_styling(
  latex_options = "HOLD_position",
  font_size = 7
)

```

Table 17: Distribution of predicate, scale, and boundedness combinations by factivity group

Factivity	Predicate + scale + boundedness	Number of trial rows	Percentage of trial rows
Canonically factive	discover + some + ub	76	1.3
	discover + some + lb	86	1.4
	discover + or + ub	101	1.7
	discover + or + lb	71	1.2
	know + some + ub	96	1.6
	know + some + lb	73	1.2
	know + or + ub	83	1.4
	know + or + lb	82	1.4
	reveal + some + ub	82	1.4
	reveal + some + lb	88	1.5
	reveal + or + ub	84	1.4
	reveal + or + lb	80	1.3
Optionally factive	prove + some + ub	70	1.2
	prove + some + lb	95	1.6
	prove + or + ub	74	1.2
	prove + or + lb	95	1.6
	confirm + some + ub	88	1.5
	confirm + some + lb	63	1.0
	confirm + or + ub	77	1.3
	confirm + or + lb	106	1.8
	establish + some + ub	81	1.3
	establish + some + lb	95	1.6
	establish + or + ub	80	1.3
	establish + or + lb	78	1.3
	acknowledge + some + ub	78	1.3
	acknowledge + some + lb	80	1.3
	acknowledge + or + ub	92	1.5
	acknowledge + or + lb	84	1.4
	hear + some + ub	84	1.4
	hear + some + lb	94	1.6
	hear + or + ub	77	1.3
	hear + or + lb	79	1.3
	inform + some + ub	93	1.5
	inform + some + lb	77	1.3
	inform + or + ub	87	1.4
	inform + or + lb	77	1.3
Non-factive	think + some + ub	68	1.1
	think + some + lb	78	1.3
	think + or + ub	100	1.7
	think + or + lb	88	1.5
	suggest + some + ub	74	1.2
	suggest + some + lb	100	1.7
	suggest + or + ub	72	1.2
	suggest + or + lb	88	1.5
	say + some + ub	112	1.9
	say + some + lb	73	1.2
	say + or + ub	75	1.2
	say + or + lb	74	1.2

D.Trial and Statement Balance Across Participants

```
# Read the final cleaned merged file
data_final <- read_csv(
  "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Cleaned data/PSE-merged.csv",
  show_col_types = FALSE
)

# Create a unique identifier for each participant
data_analysis <- data_final %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_")
  )

# Check the number of each trial type for every participant
trial_check <- data_final %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_")
  ) %>%
  group_by(participant_id) %>%
  summarise(
    total_trials = n(),
    control = sum(trigger == "control", na.rm = TRUE),
    some = sum(scale == "some", na.rm = TRUE),
    or = sum(scale == "or", na.rm = TRUE),
    ub = sum(condition == "ub", na.rm = TRUE),
    lb = sum(condition == "lb", na.rm = TRUE),
    acknowledge = sum(trigger == "acknowledge", na.rm = TRUE),
    confirm = sum(trigger == "confirm", na.rm = TRUE),
    discover = sum(trigger == "discover", na.rm = TRUE),
    establish = sum(trigger == "establish", na.rm = TRUE),
    hear = sum(trigger == "hear", na.rm = TRUE),
    inform_Sam = sum(trigger == "inform_Sam", na.rm = TRUE),
    know = sum(trigger == "know", na.rm = TRUE),
    prove = sum(trigger == "prove", na.rm = TRUE),
    reveal = sum(trigger == "reveal", na.rm = TRUE),
    say = sum(trigger == "say", na.rm = TRUE),
    suggest = sum(trigger == "suggest", na.rm = TRUE),
    think = sum(trigger == "think", na.rm = TRUE),
    .groups = "drop"
  )

# Expected number of each trial type per participant
expected_trials <- tibble(
  criterion = c(
    "total_trials", "control", "some", "or", "ub", "lb",
    "acknowledge", "confirm", "discover", "establish", "hear",
    "inform_Sam", "know", "prove", "reveal", "say", "suggest", "think"
  ),
  expected_number = c(
    18, 6, 6, 6, 6, 6,
    1, 1, 1, 1, 1, 1,
    1, 1, 1, 1, 1, 1
  )
)
```

```

)
)

# Table showing whether trial numbers are balanced across participants
trial_balance_summary <- trial_check %>%
  tidyr::pivot_longer(
    cols = -participant_id,
    names_to = "criterion",
    values_to = "number_seen"
  ) %>%
  left_join(expected_trials, by = "criterion") %>%
  group_by(criterion, expected_number) %>%
  summarise(
    minimum_seen = min(number_seen),
    maximum_seen = max(number_seen),
    participants_matching_expected = sum(number_seen == expected_number),
    .groups = "drop"
  ) %>%
  mutate(
    criterion = factor(
      criterion,
      levels = c(
        "acknowledge",
        "confirm",
        "discover",
        "establish",
        "hear",
        "inform_Sam",
        "know",
        "prove",
        "reveal",
        "say",
        "suggest",
        "think",
        "control",
        "or",
        "some",
        "lb",
        "ub",
        "total_trials"
      )
    )
  ) %>%
  arrange(criterion)

knitr::kable(
  trial_balance_summary,
  col.names = c(
    "Trial type",
    "Expected number",
    "Minimum seen",
    "Maximum seen",
    "Participants matching expected number"
  )
)

```

```

),
caption = "Trial balance across participants"
)

```

Table 18: Trial balance across participants

Trial type	Expected number	Minimum seen	Maximum seen	Participants matching expected number
acknowledge	1	1	1	334
confirm	1	1	1	334
discover	1	1	1	334
establish	1	1	1	334
hear	1	1	1	334
inform_Sam	1	1	1	334
know	1	1	1	334
prove	1	1	1	334
reveal	1	1	1	334
say	1	1	1	334
suggest	1	1	1	334
think	1	1	1	334
control	6	6	6	334
or	6	6	6	334
some	6	6	6	334
lb	6	6	6	334
ub	6	6	6	334
total_trials	18	18	18	334

```

# Count how many times each statement appeared for each participant
statement_check <- data_final %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_")
  ) %>%
  count(
    participant_id,
    statement,
    name = "times_seen"
  )

statement_summary <- statement_check %>%
  group_by(participant_id) %>%
  summarise(
    number_of_unique_statements = n(),
    highest_statement_repetition = max(times_seen),
    .groups = "drop"
  )

# Table showing whether statements were balanced across participants
statement_balance_summary <- bind_rows(

  statement_summary %>%
    summarise(
      criterion = "Unique statements",

```

```

    expected_number = 18,
    minimum_seen = min(number_of_unique_statements),
    maximum_seen = max(number_of_unique_statements),
    participants_matching_expected =
      sum(number_of_unique_statements == 18)
  ),

  statement_summary %>%
    summarise(
      criterion = "Highest repetition of one statement",
      expected_number = 1,
      minimum_seen = min(highest_statement_repetition),
      maximum_seen = max(highest_statement_repetition),
      participants_matching_expected =
        sum(highest_statement_repetition == 1)
    )
)

knitr::kable(
  statement_balance_summary,
  col.names = c(
    "Statement check",
    "Expected number",
    "Minimum seen",
    "Maximum seen",
    "Participants matching expected number"
  ),
  caption = "Statement balance across participants"
)

```

Table 19: Statement balance across participants

Statement check	Expected number	Minimum seen	Maximum seen	Participants matching expected number
Unique statements	18	18	18	334
Highest repetition of one statement	1	1	1	334

VIII.Plots

A. Mean Certainty Ratings by Predicate

```
# Read the final cleaned merged file
data_raw <- read_csv(
  "/Users/taravat/Desktop/QP/QP/Data/PSE-Main data/Cleaned data/PSE-merged.csv",
  show_col_types = FALSE
) %>%
  mutate(
    participant_id = paste(workerid, time_in_minutes, sep = "_"),
    verb = trigger
  )

# Selecting only the variables needed for Figure 1
figure1_data <- data_raw %>%
  select(
    participant_id,
    verb,
    response
  )

# Making sure responses are numeric
figure1_data <- figure1_data %>%
  mutate(
    response = as.numeric(response)
  )

# Removing missing responses
figure1_data <- figure1_data %>%
  filter(
    !is.na(response),
    !is.na(verb),
    !is.na(participant_id)
  )

# Changing inform_Sam to inform
figure1_data <- figure1_data %>%
  mutate(
    verb = recode(
      verb,
      "inform_Sam" = "inform"
    )
  )

# Categorizing predicates by factivity type
figure1_data <- figure1_data %>%
  mutate(
    predicate_type = case_when(
      verb == "control" ~ "Control",

      verb %in% c(
        "think",
```

```

      "suggest",
      "say"
    ) ~ "Nonfactive",

    verb %in% c(
      "prove",
      "confirm",
      "establish",
      "acknowledge",
      "hear",
      "inform"
    ) ~ "Optionally factive",

    verb %in% c(
      "discover",
      "know",
      "reveal"
    ) ~ "Canonically factive",

    TRUE ~ NA_character_
  )
)

# Average response within each participant and predicate
participant_predicate_data <- figure1_data %>%
  group_by(
    participant_id,
    verb,
    predicate_type
  ) %>%
  summarise(
    participant_rating = mean(
      response,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

# Mean certainty rating for each predicate
predicate_summary <- participant_predicate_data %>%
  group_by(
    verb,
    predicate_type
  ) %>%
  summarise(
    mean_certainty = mean(
      participant_rating,
      na.rm = TRUE
    ),
    number_of_participants = n(),
    .groups = "drop"
  )

```

```

# Function to calculate a 95% bootstrapped confidence interval
bootstrap_mean_ci <- function(
  values,
  number_of_bootstraps = 5000
) {

  values <- values[!is.na(values)]

  bootstrap_means <- replicate(
    number_of_bootstraps,
    mean(
      sample(
        values,
        size = length(values),
        replace = TRUE
      )
    )
  )

  tibble(
    lower_ci = quantile(
      bootstrap_means,
      probs = 0.025
    ),

    upper_ci = quantile(
      bootstrap_means,
      probs = 0.975
    )
  )
}

# Make bootstrap results reproducible
set.seed(2026)

# Calculate bootstrap confidence intervals for each predicate
predicate_bootstrap_ci <- participant_predicate_data %>%
  group_by(
    verb,
    predicate_type
  ) %>%
  group_modify(
    ~ bootstrap_mean_ci(
      .x$participant_rating,
      number_of_bootstraps = 5000
    )
  ) %>%
  ungroup()

# Combine predicate means and confidence intervals
predicate_summary <- predicate_summary %>%
  left_join(
    predicate_bootstrap_ci,

```

```

    by = c(
      "verb",
      "predicate_type"
    )
  )
)

# Order predicates from lowest to highest mean certainty
predicate_order <- predicate_summary %>%
  arrange(mean_certainty) %>%
  pull(verb) %>%
  as.character()

# Apply the same order to both plotting datasets
participant_predicate_data <- participant_predicate_data %>%
  mutate(
    verb = factor(
      as.character(verb),
      levels = predicate_order
    )
  )

predicate_summary <- predicate_summary %>%
  mutate(
    verb = factor(
      as.character(verb),
      levels = predicate_order
    )
  )

# Create Figure 1
figure1 <- ggplot(
  participant_predicate_data,
  aes(
    x = verb,
    y = participant_rating
  )
) +

# Show the distribution of participant-level ratings
geom_violin(
  aes(
    group = verb
  ),
  fill = "white",
  color = "grey80",
  linewidth = 0.6,
  width = 0.85,
  scale = "width",
  trim = FALSE
) +

# Show 95% bootstrapped confidence intervals
geom_errorbar(

```

```

data = predicate_summary,
aes(
  x = verb,
  ymin = lower_ci,
  ymax = upper_ci
),
inherit.aes = FALSE,
color = "black",
width = 0.10,
linewidth = 0.7
) +

# Show predicate means
geom_point(
  data = predicate_summary,
  aes(
    x = verb,
    y = mean_certainty,
    color = predicate_type,
    shape = predicate_type
  ),
  inherit.aes = FALSE,
  size = 3
) +

# Match colours to predicate categories
scale_color_manual(
  values = c(
    "Control" = "black",
    "Nonfactive" = "grey55",
    "Optionally factive" = "#F15A3A",
    "Canonically factive" = "#9C39C6"
  ),
  breaks = c(
    "Control",
    "Nonfactive",
    "Optionally factive",
    "Canonically factive"
  ),
  labels = c(
    "Control" = "main clause controls",
    "Nonfactive" = "nonfactive",
    "Optionally factive" = "optionally factive",
    "Canonically factive" = "factive"
  )
) +

# Match shapes to predicate categories
scale_shape_manual(
  values = c(
    "Control" = 16,
    "Nonfactive" = 15,
    "Optionally factive" = 17,

```

```

    "Canonically factive" = 18
  ),
  breaks = c(
    "Control",
    "Nonfactive",
    "Optionally factive",
    "Canonically factive"
  ),
  labels = c(
    "Control" = "main clause controls",
    "Nonfactive" = "nonfactive",
    "Optionally factive" = "optionally factive",
    "Canonically factive" = "factive"
  )
) +

# Display control as MC on the x-axis
scale_x_discrete(
  limits = predicate_order,
  labels = function(x) {
    ifelse(
      x == "control",
      "MC",
      x
    )
  }
) +

# Show ticks from 0 to 1
scale_y_continuous(
  breaks = seq(
    0,
    1,
    by = 0.2
  ),
  expand = expansion(
    mult = c(0.01, 0.03)
  )
) +

coord_cartesian(
  ylim = c(0, 1)
) +

labs(
  x = "Predicate",
  y = "Mean certainty rating",
  color = "Predicate type",
  shape = "Predicate type",
  caption = "Figure 1. Mean certainty rating by predicate."
) +

theme_classic() +

```

```

theme(
  axis.text.x = element_text(
    angle = 50,
    hjust = 1,
    vjust = 1
  ),

  legend.position = "bottom",

  plot.caption.position = "plot",

  plot.caption = element_text(
    hjust = 0,
    size = 11,
    face = "plain",
    margin = margin(
      t = 12
    )
  ),

  panel.border = element_rect(
    color = "black",
    fill = NA,
    linewidth = 0.6
  )
) +

guides(
  color = guide_legend(
    title.position = "left",
    nrow = 1
  ),

  shape = guide_legend(
    title.position = "left",
    nrow = 1
  )
)

# Display Figure 1
figure1

```

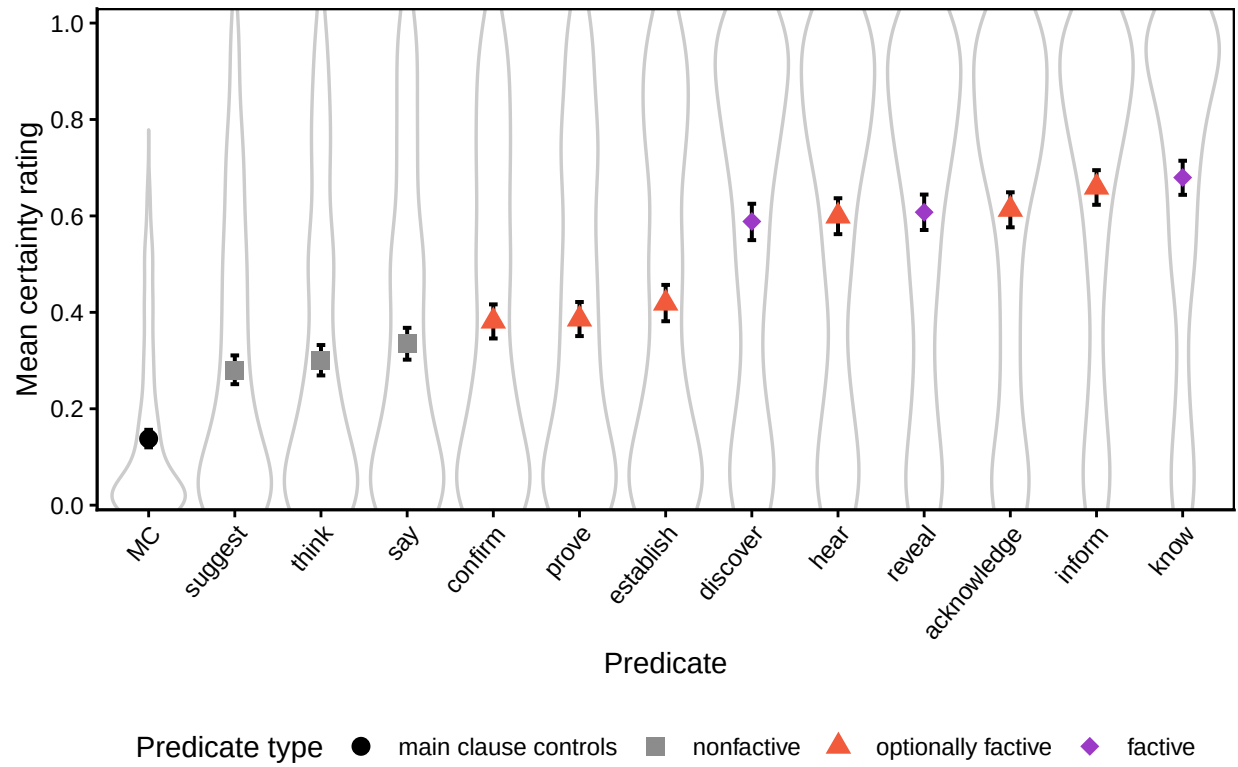


Figure 1. Mean certainty rating by predicate.

B. Mean Certainty Ratings by Experimental Statement (Item Variability)

```
# Prepare data for the statement-level figure
statement_figure_data <- data_raw %>%
  select(
    participant_id,
    statement,
    response,
    scale
  ) %>%
  mutate(
    response = as.numeric(response),

    scale_name = case_when(
      scale == "or" ~ "Disjunction",
      scale == "some" ~ "Quantifier some"
    )
  ) %>%
  filter(
    !is.na(participant_id),
    !is.na(statement),
    !is.na(response),
    !is.na(scale_name)
  )

# Order statements as requested
statement_order <- c(
  "Tony graded the midterm or the final or both",
  "Grace stole a laptop or a calculator or both",
  "Jayden solved the crossword or the Rubik's cube or both",
  "Josh drank coffee or tea or both",
  "Emily failed math or science or both",
  "Isabella ate cake or pie or both",
  "Emily failed math or science, and she did not fail both",
  "Josh drank coffee or tea, and he did not drink both",
  "Tony graded the midterm or the final, and he did not grade both",
  "Jayden solved the crossword or the Rubik's cube, and he did not solve both",
  "Grace stole a laptop or a calculator, and she did not steal both",
  "Isabella ate cake or pie, and she did not eat both",

  "Frank graded at least some of the math problems",
  "Emma stole at least some of the office supplies",
  "Josie failed at least some of the courses",
  "Jackson drank at least some of the sodas",
  "Mary ate at least some of the cupcakes",
  "Danny solved at least some of the puzzles",
  "Mary ate some of the cupcakes, and she did not eat all of them",
  "Jackson drank some of the sodas, and he did not drink all of them",
  "Josie failed some of the courses, and she did not fail all of them",
  "Frank graded some of the math problems, and he did not grade all of them",
  "Emma stole some of the office supplies, and she did not take all of them",
  "Danny solved some of the puzzles, and he did not solve all of them"
)
```

```

# Create coloured x-axis labels
statement_labels <- vapply(
  seq_along(statement_order),
  function(i) {

    statement_color <- ifelse(
      ((i - 1) %% 12) + 1 <= 6,
      "#006400",
      "#8B0000"
    )

    wrapped_statement <- paste(
      strwrap(
        statement_order[i],
        width = 28
      ),
      collapse = "<br>"
    )

    paste0(
      "<span style='color:",
      statement_color,
      ">",
      wrapped_statement,
      "</span>"
    )
  },
  character(1)
)

names(statement_labels) <- statement_order

# Calculate each participant's rating for each statement
participant_statement_data <- statement_figure_data %>%
  group_by(
    participant_id,
    statement,
    scale_name
  ) %>%
  summarise(
    participant_rating = mean(
      response,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

# Calculate mean certainty rating for each statement
statement_summary <- participant_statement_data %>%
  group_by(
    statement,
    scale_name
  ) %>%

```

```

summarise(
  mean_certainty = mean(
    participant_rating,
    na.rm = TRUE
  ),
  .groups = "drop"
)

# Calculate 95% bootstrap confidence intervals
set.seed(2027)

statement_bootstrap_ci <- participant_statement_data %>%
  group_by(
    statement,
    scale_name
  ) %>%
  group_modify(
    ~ bootstrap_mean_ci(
      .x$participant_rating,
      number_of_bootstraps = 5000
    )
  ) %>%
  ungroup()

# Combine means and confidence intervals
statement_summary <- statement_summary %>%
  left_join(
    statement_bootstrap_ci,
    by = c(
      "statement",
      "scale_name"
    )
  )

# Apply the statement order
participant_statement_data <- participant_statement_data %>%
  mutate(
    statement = factor(
      statement,
      levels = statement_order
    )
  )

statement_summary <- statement_summary %>%
  mutate(
    statement = factor(
      statement,
      levels = statement_order
    )
  )

# Create the statement-level plot
statement_figure <- ggplot(

```

```

participant_statement_data,
aes(
  x = statement,
  y = participant_rating
)
) +

geom_violin(
  aes(
    group = statement
  ),
  fill = "white",
  color = "grey80",
  linewidth = 0.5,
  width = 0.85,
  scale = "width",
  trim = FALSE
) +

geom_errorbar(
  data = statement_summary,
  aes(
    x = statement,
    ymin = lower_ci,
    ymax = upper_ci
  ),
  inherit.aes = FALSE,
  color = "black",
  width = 0.08,
  linewidth = 0.6
) +

geom_point(
  data = statement_summary,
  aes(
    x = statement,
    y = mean_certainty
  ),
  inherit.aes = FALSE,
  color = "black",
  size = 2.5
) +

scale_x_discrete(
  labels = statement_labels
) +

scale_y_continuous(
  breaks = seq(
    0,
    1,
    by = 0.2
  ),

```

```

    expand = expansion(
      mult = c(
        0.01,
        0.03
      )
    )
  ) +

  coord_cartesian(
    ylim = c(
      0,
      1
    )
  ) +

  facet_wrap(
    ~ scale_name,
    ncol = 1,
    scales = "free_x"
  ) +

  labs(
    x = "Statement",
    y = "Mean certainty rating",
    caption = "Figure 2. Mean certainty ratings by experimental statement."
  ) +

  theme_classic() +

  theme(
    axis.text.x = ggtext::element_markdown(
      angle = 45,
      hjust = 1,
      vjust = 1,
      size = 9
    ),
    axis.title = element_text(
      size = 12
    ),
    axis.text.y = element_text(
      size = 10
    ),
    strip.text = element_text(
      size = 12,
      face = "bold"
    ),
    plot.caption.position = "plot",
    plot.caption = element_text(

```

```

    hjust = 0,
    size = 10,
    margin = margin(
      t = 12
    )
  ),

  plot.margin = margin(
    t = 8,
    r = 12,
    b = 8,
    l = 20
  ),

  panel.border = element_rect(
    color = "black",
    fill = NA,
    linewidth = 0.6
  )
)

# Display Figure 2
statement_figure

```

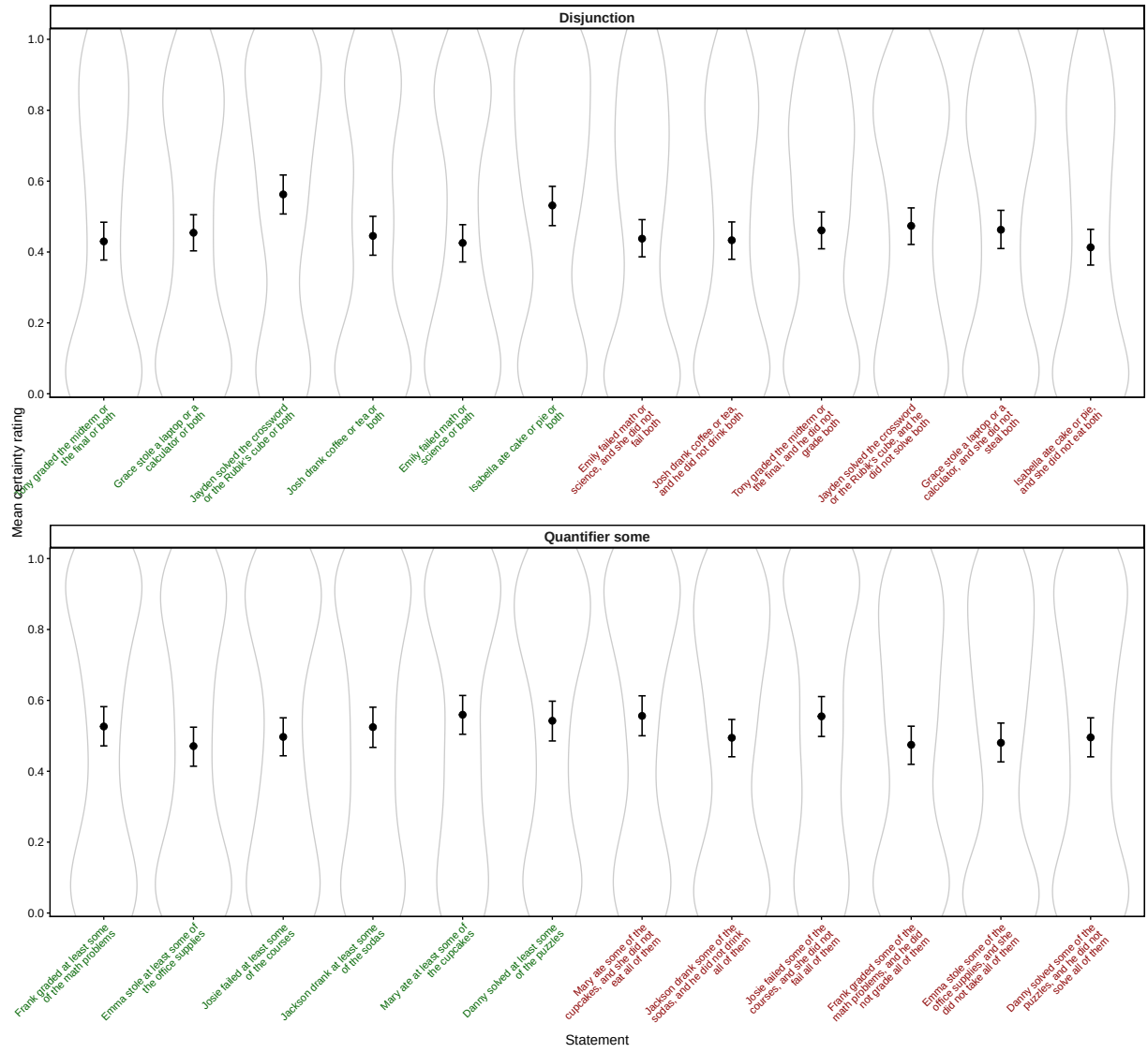


Figure 2. Mean certainty ratings by experimental statement.

C. Mean Certainty Ratings by Predicate and Boundedness

```
# Selecting the variables needed for Figure 2
figure3_data <- data_raw %>%
  select(
    participant_id,
    verb,
    response,
    condition,
    scale
  )

# Making sure response is numeric
figure3_data <- figure3_data %>%
  mutate(
    response = as.numeric(response)
  )

# Removing rows without a response, predicate, or participant ID
figure3_data <- figure3_data %>%
  filter(
    !is.na(response),
    !is.na(verb),
    !is.na(participant_id)
  )

# Changing inform_Sam to inform
figure3_data <- figure3_data %>%
  mutate(
    verb = recode(
      verb,
      "inform_Sam" = "inform"
    )
  )

# Categorizing predicates by predicate type
figure3_data <- figure3_data %>%
  mutate(
    predicate_type = case_when(
      verb == "control" ~ "Control",

      verb %in% c(
        "think",
        "suggest",
        "say"
      ) ~ "Nonfactive",

      verb %in% c(
        "prove",
        "confirm",
        "establish",
        "acknowledge",
        "hear",

```

```

      "inform"
    ) ~ "Optionally factive",

    verb %in% c(
      "discover",
      "know",
      "reveal"
    ) ~ "Canonically factive",

    TRUE ~ NA_character_
  )
)

# Coding boundedness from the condition column
figure3_data <- figure3_data %>%
  mutate(
    boundedness = case_when(
      verb == "control" ~ "Control",
      condition == "ub" ~ "Upper-bounded",
      condition == "lb" ~ "Lower-bounded",
      TRUE ~ NA_character_
    )
  )

# Removing predicates or trials that were not successfully categorized
figure3_data <- figure3_data %>%
  filter(
    !is.na(predicate_type),
    !is.na(boundedness)
  )

# Calculating each participant's mean rating
# for each predicate and boundedness condition
participant_boundedness_data <- figure3_data %>%
  group_by(
    participant_id,
    verb,
    predicate_type,
    boundedness
  ) %>%
  summarise(
    participant_rating = mean(
      response,
      na.rm = TRUE
    ),
    .groups = "drop"
  )

# Calculating the overall mean for each predicate and boundedness condition
boundedness_summary <- participant_boundedness_data %>%
  group_by(
    verb,
    predicate_type,

```

```

    boundedness
  ) %>%
  summarise(
    mean_certainty = mean(
      participant_rating,
      na.rm = TRUE
    ),
    number_of_participants = n(),
    .groups = "drop"
  )

# Making bootstrap results reproducible
set.seed(2026)

# Calculating 95% bootstrap confidence intervals
boundedness_bootstrap_ci <- participant_boundedness_data %>%
  group_by(
    verb,
    predicate_type,
    boundedness
  ) %>%
  group_modify(
    ~ bootstrap_mean_ci(
      .x$participant_rating,
      number_of_bootstraps = 5000
    )
  ) %>%
  ungroup()

# Combining means and confidence intervals
boundedness_summary <- boundedness_summary %>%
  left_join(
    boundedness_bootstrap_ci,
    by = c(
      "verb",
      "predicate_type",
      "boundedness"
    )
  )

# Creating the predicate order based only on upper-bounded means
upper_bounded_order <- boundedness_summary %>%
  filter(
    boundedness == "Upper-bounded"
  ) %>%
  arrange(
    mean_certainty
  ) %>%
  pull(
    verb
  ) %>%
  as.character()

```

```

# Putting the main-clause control first
figure3_predicate_order <- c(
  "control",
  upper_bounded_order
)

# Removing possible duplicate predicate names
figure3_predicate_order <- unique(
  figure3_predicate_order
)

# Applying the order to participant-level data
participant_boundedness_data <- participant_boundedness_data %>%
  mutate(
    verb = factor(
      as.character(verb),
      levels = figure3_predicate_order
    )
  )

# Applying the same order to summary data
boundedness_summary <- boundedness_summary %>%
  mutate(
    verb = factor(
      as.character(verb),
      levels = figure3_predicate_order
    )
  )

# Creating a color category for the plotted points
boundedness_summary <- boundedness_summary %>%
  mutate(
    point_group = case_when(
      boundedness == "Control" ~
        "Main clause controls",

      boundedness == "Lower-bounded" ~
        "Lower-bounded",

      boundedness == "Upper-bounded" &
        predicate_type == "Nonfactive" ~
        "Upper-bounded: nonfactive",

      boundedness == "Upper-bounded" &
        predicate_type == "Optionally factive" ~
        "Upper-bounded: optionally factive",

      boundedness == "Upper-bounded" &
        predicate_type == "Canonically factive" ~
        "Upper-bounded: factive",

      TRUE ~ NA_character_
    )
  )

```

```

)

# Creating Figure 3
figure3 <- ggplot(
  participant_boundedness_data,
  aes(
    x = verb,
    y = participant_rating
  )
) +

# Showing one participant-level distribution for each predicate
geom_violin(
  aes(
    group = verb
  ),
  fill = "white",
  color = "grey80",
  linewidth = 0.6,
  width = 0.85,
  scale = "width",
  trim = FALSE
) +

# Showing 95% bootstrap confidence intervals
geom_errorbar(
  data = boundedness_summary,
  aes(
    x = verb,
    ymin = lower_ci,
    ymax = upper_ci,
    color = point_group
  ),
  inherit.aes = FALSE,
  width = 0.03,
  linewidth = 0.3
) +

# Showing lower-bounded means
geom_point(
  data = boundedness_summary %>%
    filter(
      boundedness == "Lower-bounded"
    ),
  aes(
    x = verb,
    y = mean_certainty,
    color = point_group,
    shape = predicate_type
  ),
  inherit.aes = FALSE,
  size = 1.5
) +

```

```

# Showing control and upper-bounded means
geom_point(
  data = boundedness_summary %>%
    filter(
      boundedness != "Lower-bounded"
    ),
  aes(
    x = verb,
    y = mean_certainty,
    color = point_group,
    shape = predicate_type
  ),
  inherit.aes = FALSE,
  size = 3.0
) +

# Keep the existing colour coding
scale_color_manual(
  values = c(
    "Main clause controls" = "black",
    "Upper-bounded: nonfactive" = "grey55",
    "Upper-bounded: optionally factive" = "#F15A3A",
    "Upper-bounded: factive" = "#9C39C6",
    "Lower-bounded" = "#2E8B57"
  ),

  breaks = c(
    "Main clause controls",
    "Upper-bounded: nonfactive",
    "Upper-bounded: optionally factive",
    "Upper-bounded: factive",
    "Lower-bounded"
  )
) +

# Keep the same predicate-type shapes as Figure 1
scale_shape_manual(
  values = c(
    "Control" = 16,
    "Nonfactive" = 15,
    "Optionally factive" = 17,
    "Canonically factive" = 18
  ),

  breaks = c(
    "Control",
    "Nonfactive",
    "Optionally factive",
    "Canonically factive"
  ),

  labels = c(
    "Control" = "main clause controls",

```

```

    "Nonfactive" = "nonfactive",
    "Optionally factive" = "optionally factive",
    "Canonically factive" = "factive"
  )
) +

# Display control as MC on the x-axis
scale_x_discrete(
  limits = figure3_predicate_order,

  labels = function(x) {
    ifelse(
      x == "control",
      "MC",
      x
    )
  }
) +

scale_y_continuous(
  breaks = seq(
    0,
    1,
    by = 0.2
  ),

  expand = expansion(
    mult = c(
      0.01,
      0.03
    )
  )
) +

coord_cartesian(
  ylim = c(
    0,
    1
  )
) +

labs(
  x = "Predicate",
  y = "Mean certainty rating",
  color = "Boundedness and predicate type",
  shape = "Predicate type",
  caption = "Figure 3. Mean certainty ratings by predicate and boundedness condition."
) +

theme_classic() +

theme(
  axis.text.x = element_text(

```

```

    angle = 50,
    hjust = 1,
    vjust = 1
  ),

  axis.title = element_text(size = 12),
  axis.text = element_text(size = 10),

  legend.position = "bottom",
  legend.box = "vertical",
  legend.title = element_text(size = 9),
  legend.text = element_text(size = 8),
  legend.spacing.x = unit(0.15, "cm"),
  legend.spacing.y = unit(0.05, "cm"),
  legend.key.width = unit(0.6, "cm"),
  legend.key.height = unit(0.4, "cm"),

  plot.caption.position = "plot",
  plot.caption = element_text(
    hjust = 0,
    size = 10,
    face = "plain",
    margin = margin(t = 12)
  ),

  plot.margin = margin(
    t = 5,
    r = 10,
    b = 5,
    l = 18
  ),

  panel.border = element_rect(
    color = "black",
    fill = NA,
    linewidth = 0.6
  )
)+

guides(
  color = guide_legend(
    title.position = "top",
    nrow = 2,
    order = 1
  ),

  shape = guide_legend(
    title.position = "top",
    nrow = 1,
    order = 2
  )
)
# Display Figure 3

```

figure3

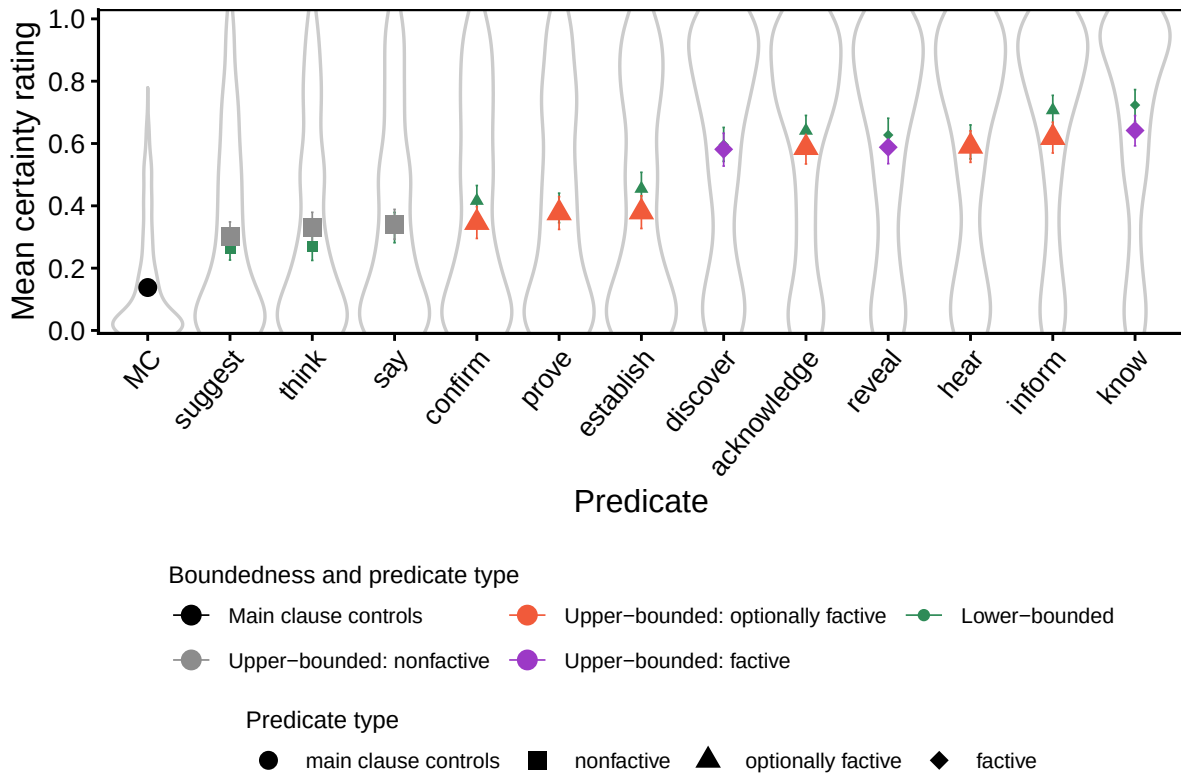


Figure 3. Mean certainty ratings by predicate and boundedness condition.

```

# Prepare data for Figures 4 and 5
figure45_data <- figure3_data %>%
  transmute(
    participant_id,
    verb = as.character(verb),
    response = as.numeric(response),
    predicate_type,

    scale = case_when(
      verb == "control" ~ "Control",
      scale == "or" ~ "Disjunction",
      scale == "some" ~ "Some",
      TRUE ~ NA_character_
    ),

    boundedness = case_when(
      verb == "control" ~ "Control",
      condition == "ub" ~ "Upper-bounded",
      condition == "lb" ~ "Lower-bounded",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(
    !is.na(participant_id),
    !is.na(verb),
    !is.na(response),
    !is.na(predicate_type),
    !is.na(scale),
    !is.na(boundedness)
  ) %>%
  mutate(
    scale = factor(
      scale,
      levels = c(
        "Control",
        "Disjunction",
        "Some"
      )
    ),

    boundedness = factor(
      boundedness,
      levels = c(
        "Control",
        "Upper-bounded",
        "Lower-bounded"
      )
    )
  )

# Function for preparing and plotting one scale
make_boundedness_scale_plot <- function(
  data,

```

```

    selected_scale,
    figure_number,
    caption_scale
  ) {

    # Keep one scale and the main-clause controls
    scale_plot_data <- data %>%
      filter(
        scale %in% c(
          "Control",
          selected_scale
        )
      )

    # Calculate participant-level means
    participant_scale_data <- scale_plot_data %>%
      group_by(
        participant_id,
        verb,
        predicate_type,
        boundedness
      ) %>%
      summarise(
        participant_rating = mean(
          response,
          na.rm = TRUE
        ),
        .groups = "drop"
      )

    # Calculate means for each predicate and boundedness condition
    scale_summary <- participant_scale_data %>%
      group_by(
        verb,
        predicate_type,
        boundedness
      ) %>%
      summarise(
        mean_certainty = mean(
          participant_rating,
          na.rm = TRUE
        ),

        number_of_participants = n_distinct(
          participant_id
        ),

        .groups = "drop"
      )

    # Calculate 95% bootstrap confidence intervals
    set.seed(
      2026 + figure_number

```

```

)

scale_bootstrap_ci <- participant_scale_data %>%
  group_by(
    verb,
    predicate_type,
    boundedness
  ) %>%
  group_modify(
    ~ bootstrap_mean_ci(
      .x$participant_rating,
      number_of_bootstraps = 5000
    )
  ) %>%
  ungroup()

# Combine means and confidence intervals
scale_summary <- scale_summary %>%
  left_join(
    scale_bootstrap_ci,
    by = c(
      "verb",
      "predicate_type",
      "boundedness"
    )
  )

# Order predicates by their upper-bounded means
upper_mean_table <- scale_summary %>%
  filter(
    verb != "control",
    boundedness == "Upper-bounded"
  ) %>%
  select(
    verb,
    upper_mean = mean_certainty
  )

predicate_order_table <- scale_plot_data %>%
  filter(
    verb != "control"
  ) %>%
  distinct(
    verb
  ) %>%
  left_join(
    upper_mean_table,
    by = "verb"
  ) %>%
  arrange(
    is.na(upper_mean),
    upper_mean,
    verb
  )

```

```

)

predicate_order <- c(
  "control",
  predicate_order_table$verb
)

predicate_order <- unique(
  predicate_order
)

# Apply the order to participant-level data
participant_scale_data <- participant_scale_data %>%
  mutate(
    verb = factor(
      as.character(verb),
      levels = predicate_order
    )
  )

# Apply the same order to summary data
scale_summary <- scale_summary %>%
  mutate(
    verb = factor(
      as.character(verb),
      levels = predicate_order
    )
  )

# Create colour groups for the plotted points
scale_summary <- scale_summary %>%
  mutate(
    point_group = case_when(
      boundedness == "Control" ~
        "Main clause controls",

      boundedness == "Lower-bounded" ~
        "Lower-bounded",

      boundedness == "Upper-bounded" &
        predicate_type == "Nonfactive" ~
        "Upper-bounded: nonfactive",

      boundedness == "Upper-bounded" &
        predicate_type == "Optionally factive" ~
        "Upper-bounded: optionally factive",

      boundedness == "Upper-bounded" &
        predicate_type == "Canonically factive" ~
        "Upper-bounded: factive",

      TRUE ~ NA_character_
    )
  )

```

```

)

# Create the plot
scale_plot <- ggplot(
  participant_scale_data,
  aes(
    x = verb,
    y = participant_rating
  )
) +

  geom_violin(
    aes(
      group = verb
    ),
    fill = "white",
    color = "grey80",
    linewidth = 0.6,
    width = 0.85,
    scale = "width",
    trim = FALSE
  ) +

  geom_errorbar(
    data = scale_summary,
    aes(
      x = verb,
      ymin = lower_ci,
      ymax = upper_ci,
      color = point_group
    ),
    inherit.aes = FALSE,
    width = 0.03,
    linewidth = 0.35
  ) +

  geom_point(
    data = scale_summary %>%
      filter(
        boundedness == "Lower-bounded"
      ),
    aes(
      x = verb,
      y = mean_certainty,
      color = point_group,
      shape = predicate_type
    ),
    inherit.aes = FALSE,
    size = 2
  ) +

  geom_point(
    data = scale_summary %>%

```

```

    filter(
      boundedness != "Lower-bounded"
    ),
    aes(
      x = verb,
      y = mean_certainty,
      color = point_group,
      shape = predicate_type
    ),
    inherit.aes = FALSE,
    size = 3
  ) +

  scale_color_manual(
    values = c(
      "Main clause controls" = "black",
      "Upper-bounded: nonfactive" = "grey55",
      "Upper-bounded: optionally factive" = "#F15A3A",
      "Upper-bounded: factive" = "#9C39C6",
      "Lower-bounded" = "#2E8B57"
    ),

    breaks = c(
      "Main clause controls",
      "Upper-bounded: nonfactive",
      "Upper-bounded: optionally factive",
      "Upper-bounded: factive",
      "Lower-bounded"
    ),

    name = "Boundedness and predicate type"
  ) +

  scale_shape_manual(
    values = c(
      "Control" = 16,
      "Nonfactive" = 15,
      "Optionally factive" = 17,
      "Canonically factive" = 18
    ),

    breaks = c(
      "Control",
      "Nonfactive",
      "Optionally factive",
      "Canonically factive"
    ),

    labels = c(
      "Control" = "main clause controls",
      "Nonfactive" = "nonfactive",
      "Optionally factive" = "optionally factive",
      "Canonically factive" = "factive"
    )
  )

```

```

    ),

    name = "Predicate type"
) +

scale_x_discrete(
  limits = predicate_order,

  labels = function(x) {
    ifelse(
      x == "control",
      "MC",
      x
    )
  },

  drop = FALSE
) +

scale_y_continuous(
  breaks = seq(
    0,
    1,
    by = 0.2
  ),

  expand = expansion(
    mult = c(
      0.01,
      0.03
    )
  )
) +

coord_cartesian(
  ylim = c(
    0,
    1
  )
) +

labs(
  x = "Predicate",
  y = "Mean certainty rating",

  caption = paste0(
    "Figure ",
    figure_number,
    ". Mean certainty ratings by predicate and boundedness ",
    "for the ",
    caption_scale,
    " scale."
  )
)

```

```

) +

theme_classic() +

theme(
  axis.text.x = element_text(
    angle = 50,
    hjust = 1,
    vjust = 1
  ),

  axis.title = element_text(
    size = 12
  ),

  axis.text = element_text(
    size = 10
  ),

  legend.title = element_text(
    size = 9
  ),

  legend.text = element_text(
    size = 8
  ),

  legend.position = "bottom",
  legend.box = "vertical",

  legend.spacing.x = grid::unit(
    0.15,
    "cm"
  ),

  legend.spacing.y = grid::unit(
    0.05,
    "cm"
  ),

  legend.key.width = grid::unit(
    0.6,
    "cm"
  ),

  legend.key.height = grid::unit(
    0.4,
    "cm"
  ),

  plot.caption.position = "plot",
  plot.caption = element_text(

```

```

      hjust = 0,
      size = 10,
      face = "plain",
      margin = margin(
        t = 12,
        b = 8
      )
    ),

    plot.margin = margin(
      t = 8,
      r = 12,
      b = 14,
      l = 20
    ),

    panel.border = element_rect(
      color = "black",
      fill = NA,
      linewidth = 0.6
    )
  ) +

  guides(
    color = guide_legend(
      title.position = "top",
      nrow = 2,
      order = 1
    ),

    shape = guide_legend(
      title.position = "top",
      nrow = 1,
      order = 2
    )
  )

list(
  plot = scale_plot,
  participant_data = participant_scale_data,
  summary = scale_summary,
  predicate_order = predicate_order
)
}

```

D. Mean Certainty Ratings by Predicate, Boundedness, and Disjunction

```
figure4_results <- make_boundedness_scale_plot(
  data = figure45_data,
  selected_scale = "Disjunction",
  figure_number = 4,
  caption_scale = "disjunction"
)

figure4 <- figure4_results$plot

figure4
```

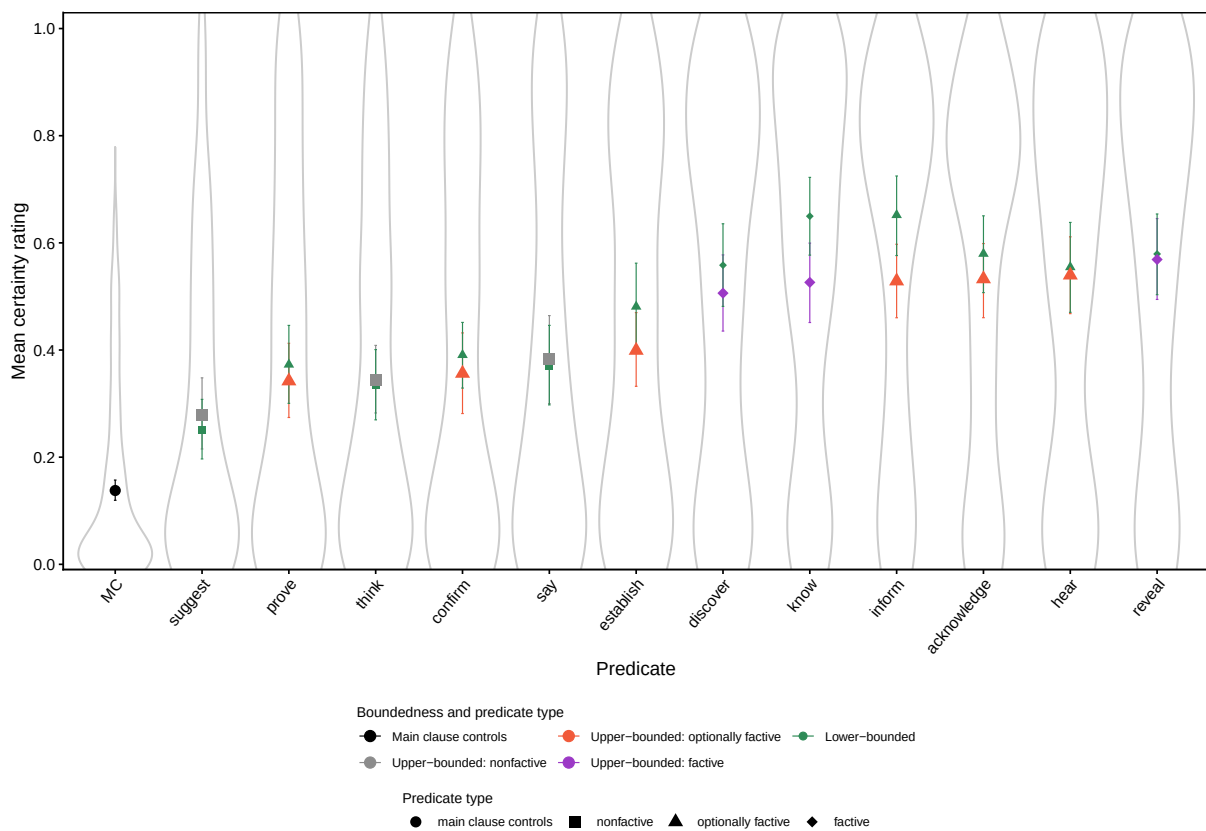


Figure 4. Mean certainty ratings by predicate and boundedness for the disjunction scale.

E. Mean Certainty Ratings by Predicate, Boundedness, and the Quantifier *some*

```
figure5_results <- make_boundedness_scale_plot(
  data = figure45_data,
  selected_scale = "Some",
  figure_number = 5,
  caption_scale = "quantifier 'some'"
)

figure5 <- figure5_results$plot

figure5
```

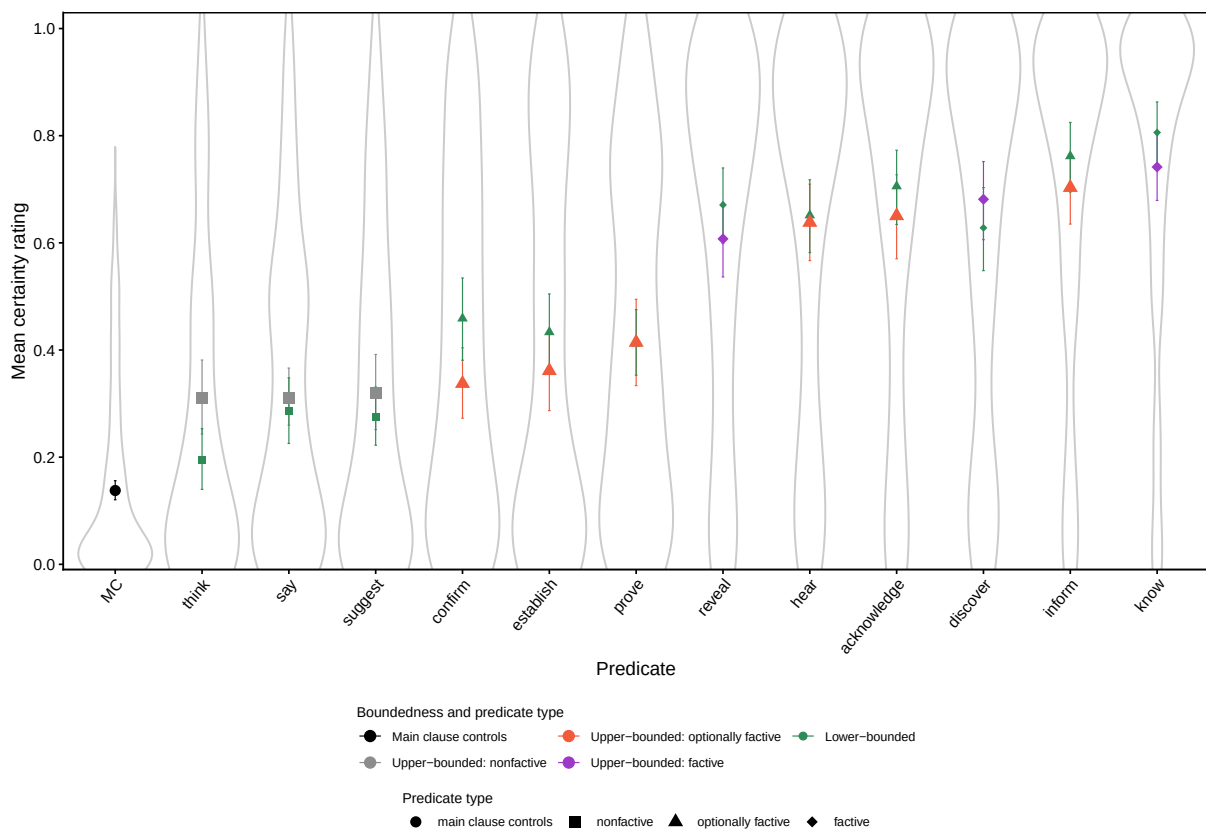


Figure 5. Mean certainty ratings by predicate and boundedness for the quantifier 'some' scale.

F. Combined version of Figures 4 and 5

```
# Combined version of Figures 4 and 5
figure4_and_figure5 <- patchwork::wrap_plots(
  plots = list(
    figure4,
    figure5,
    patchwork::guide_area()
  ),

  ncol = 1,

  heights = c(
    1,
    1,
    0.25
  ),

  guides = "collect"
) &
ggplot2::theme(
  legend.position = "bottom",
  legend.box = "vertical",
  legend.box.just = "center",

  plot.margin = ggplot2::margin(
    t = 8,
    r = 16,
    b = 5,
    l = 20
  )
)

figure4_and_figure5
```

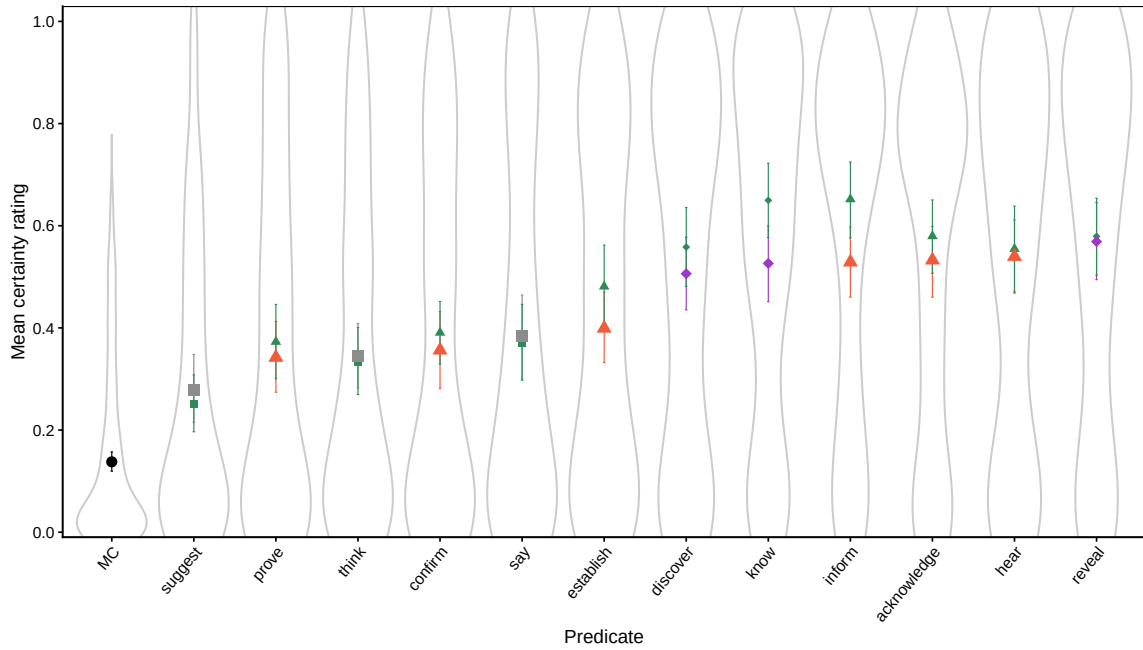


Figure 4. Mean certainty ratings by predicate and boundedness for the disjunction scale.

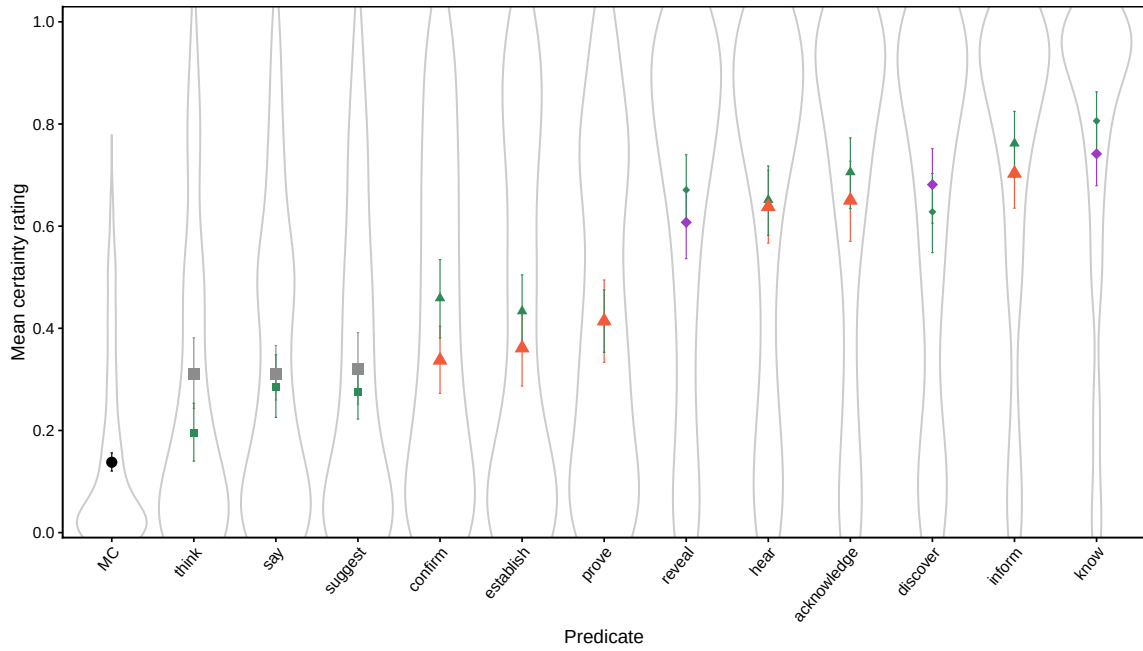


Figure 5. Mean certainty ratings by predicate and boundedness for the quantifier 'some' scale.

